

A Weighted Ensemble Approach to Semantic Similarity Using TGD

A Thesis
Presented to
The Faculty of the College of Computing and Informatics
Iloilo Science and Technology University
Lapaz, Iloilo City

In Partial Fulfilment
of the Requirements for the Degree
Bachelor of Science in Computer Science

by

Cyra Grace B. Bedoy
Angel Mariquet E. Cachuela
Julliana Antonette D. Hurgo
Mary Angel B. Palmares
Jean Grace G. Samalburo

May 2026

ILOILO SCIENCE AND TECHNOLOGY UNIVERSITY
COLLEGE OF COMPUTING AND INFORMATICS
La Paz, Iloilo City

APPROVAL SHEET

This thesis entitled “**A WEIGHTED ENSEMBLE APPROACH TO SEMANTIC SIMILARITY USING TGD**”, prepared and submitted by CYRA GRACE B. BEDOY, ANGEL MARIQUET E. CAHUELA, JULLIANA ANTONETTE D. HURGO, MARY ANGEL B. PALMARES, and JEAN GRACE G. SAMALBURO in partial fulfillment of the requirements for the degree BACHELOR OF SCIENCE IN COMPUTER SCIENCE is hereby approved.

JOYCE F. JAMILE, MSCS
Adviser

It passed the final defense on March 10, 2026 and was approved by the Defense Committee on _____ with a grade of PASS.

PANEL OF EXAMINERS

MAUREEN NETTIE N. LINAN, DIT
Member

MICHELLE P. ESCRIBA, MSCS
Member

YVETTE G. GONZALES, D. Eng
Chairperson

Accepted and approved in partial fulfillment of the requirements for the degree BACHELOR OF SCIENCE IN COMPUTER SCIENCE.

ERNEST ANDREIGH C. CENTINA, MSCS
Head, Computer Science Department
Date: _____

TRACY N. TACUBAN, DIT, PhD
Dean, College of Computing and Informatics
Date: _____

ACKNOWLEDGEMENT

The researchers would like to thank the **Almighty God** for the strength, wisdom, and guidance granted to them in completing this research work. Sincere appreciation is also extended to the individuals who supported the researchers throughout this research journey.

Mr. and Mrs. Bedoy, Mr. and Mrs. Cachuela, Mr. and Mrs. Hurgo, Mr. and Mrs. Palmares, and Mr. and Mrs. Samalburo, the researchers' parents, for their prayers, encouragement, and continuous support.

Ms. Joyce F. Jamile, thesis adviser, for her guidance, patience, and valuable insights that helped improve this research.

Dr. Maureen Nettie N. Linan, subject adviser, for her support and academic guidance during the conduct of the study.

Dr. Yvette G. Gonzales and Ms. Michelle P. Escriba, members of the Committee on Final Defense, for their constructive comments and recommendations. And to everyone who contributed in one way or another to the completion of this research, the researchers extend their sincere gratitude.

To God be the Glory!

C.G.B.B.
A.M.E.C.
J.A.D.H.
M.A.B.P.
J.G.G.S

A WEIGHTED ENSEMBLE APPROACH TO SEMANTIC SIMILARITY USING TGD

Cyra Grace B. Bedoy
Angel Mariquet E. Cachuela
Julliana Antonette D. Hurgo
Mary Angel B. Palmares
Jean Grace G. Samalburu

Joyce F. Jamile, MSCS
Adviser

ABSTRACT

Semantic textual similarity (STS) is a fundamental task in natural language processing that measures the degree of meaning shared between two sentences. The effectiveness of STS largely depends on the quality of text representation. Traditional lexical approaches, such as Term Frequency–Inverse Document Frequency (TF-IDF), efficiently capture term importance but lack semantic understanding, whereas static word embeddings, such as Global Vectors for Word Representation (GloVe),⁹ capture global semantic relationships but fail to account for contextual variations. Contextual models provide deeper contextual representations; however, their embeddings may exhibit anisotropic properties and reduced effectiveness when used without task-specific fine-tuning. To address these limitations, this study proposes a weighted ensemble approach called TGD that integrates TF-IDF, GloVe, and DeBERTa representations. Cosine similarity scores are computed independently for each embedding method and combined into a unified feature vector. A Ridge Regression model is then applied to learn the optimal weighted combination of these similarity features for predicting semantic similarity. The model was evaluated using the normalized Semantic Textual Similarity Benchmark (STS-B) dataset, where similarity scores are scaled between 0 and 1 to align with cosine similarity measures. Experimental results achieved a Pearson correlation coefficient of 0.6697, with 75.78% accuracy and an F1-score of 80.80%, demonstrating balanced predictive performance. The findings indicate that integrating lexical, global semantic, and contextual representations significantly improves semantic similarity prediction compared with standalone models.

TABLE OF CONTENTS

Approval Sheet	ii
Acknowledgement	iii
Abstract	iv
List of Tables	vii
List of Figures	viii
List of Equations	x
Chapter	
I. INTRODUCTION	
A. Background of the Study	1
B. Objectives of the Study	3
C. Conceptual Framework	4
D. Definition of Terms	5
E. Significance of the Study	11
F. Scope and Limitation of the Study	12
II. REVIEW OF RELATED LITERATURE	
A. TF-IDF	16
B. GloVe	20
C. DeBERTa	24
D. Ridge Regression	27
E. Cosine Similarity	31
F. Metrics	34
III. METHODOLOGY	
A. Research Design	38
B. Data Collection	43
C. Data Preprocessing	45
D. Machine Learning Models	45
E. Model Training and Evaluation	47
F. Evaluation Metrics	47

G. Tools and Environments	48
H. Ethical Considerations	50
IV. RESULTS AND DISCUSSION	
A. Generate TF-IDF scores, GloVe embeddings, and DeBERTa embeddings to capture term importance, global semantic meaning, and deep contextual information from the text	51
B. Compute cosine similarity scores for each embedding method	59
C. Form a feature vector from the three similarity scores	62
D. Train a Ridge Regression model to learn the optimal weighted combination of the similarity scores for predicting semantic similarity	64
E. Evaluate the ensemble model against standalone TF-IDF, GloVe, and DeBERTa models using MSE, MAE, Pearson Correlation, Accuracy, and F1-score	67
V. SUMMARY OF FINDINGS, CONCLUSIONS, RECOMMENDATIONS	
A. Summary	77
B. Conclusions	78
C. Recommendations	79
REFERENCES	
APPENDICES	
A. Implementation Code Snippets and Results	86
B. Organizational Chart	100
C. Researcher's Profile	102
D. Grammarian Certificate	108

Table		Page
I	Sample sentence pairs with cosine similarities and actual scores	33
II	A conventional approach to interpreting a correlation coefficient	35
III	Attributes of the STS-B dataset used in this study	44
IV	Similarity scores with explanations and examples	44
V	Static word embeddings comparison table	46
VI	Contextual embeddings comparison table	46
VII	Evaluation performance of standalone and ensemble models	67
VIII	Confusion matrix	73
IX	Hyperparameter Search Space per Training Settings	87
X	Best-Performing Hyperparameter Configuration	88

Figure		Page
1	Conceptual Framework	4
2	Pipeline for Extracting Features from Reference and Input Text Pairs	18
3	Model Performance Evaluation Indices and Calculation Formulas	37
4	Research Design	39
5	Feature Vector Extraction TF-IDF, GloVe, and DeBERTa Representation per Sentence Pair	41
6	Sentence Pairs and Similarity Scores (STS-B Training Split)	52
7	Sentence Pairs with Similarity Scores (STS-B Development Split)	52
8	Sentence Pairs with Similarity Scores (STS-B Test Split)	52
9	Preprocessed Dataset Overview	53
10	Code Snippet for Generating TF-IDF Vectors	53
11	TF-IDF Vectors of the First 10 Sentence Pairs from the Test Split Shown in Figure 8	54
12	Code Snippet for Generating GloVe Embeddings	55
13	GloVe Sentence Embeddings of the First 5 Sentence Pairs from the Test Split Shown in Figure 8	56
14	Code Snippet for Generating DeBERTa Embeddings	58
15	DeBERTa Sentence Embeddings of the First 5 Sentence Pairs from the Test Split Shown in Figure 8	59
16	Code Snippet for Computing Cosine Similarity using TF-IDF, GloVe, and DeBERTa Embeddings	60
17	Cosine Similarity Scores of the First 5 Sentence Pairs from the Test Split Shown in Figure 8	60
18	Code Snippet for Constructing the Combined Feature Vector from TF-IDF, GloVe, and DeBERTa Similarity Scores	62
19	Feature Vector of the First 5 Sentence Pairs (Test Split)	63
20	Code Snippet for Training a Ridge Regression Model	65
21	Training and Validation Loss and MAE per Epoch TGD (Ensemble)	66
22	Training and Validation Loss (MSE) Curves of the TGD (Ensemble)	66

23	Training and Validation Mean Absolute Error (MAE) Curves of the TGD (Ensemble)	67
24	Binary Label Representation of TF-IDF Similarity Scores	71
25	Binary Label Representation of GloVe Similarity Scores	72
26	Binary Label Representation of DeBERTa Similarity Scores	72
27	Binary Label Representation of TGD (Ensemble) Similarity Scores	72
28	Test Split Mean Squared Error (MSE) and Mean Absolute Error (MAE) of the TGD (Ensemble) Model	74
29	Correlation Scatter Plots of Predicted vs. Actual Similarity Scores	75
30	Code Snippet for Preprocessing	86
31	Code Snippet for Hyperparameter Setting	87
32	Code Snippets for the Training Experiment	92
33	Sample Output of Training Data Stored as CSV	92
34	Code Snippet for Semantic Similarity Binarization and Threshold Tuning	93
35	Confusion Matrix	94
36	TGD Dashboard Home Page	94
37	TGD Dashboard Dataset Page	94
38	TGD Dashboard Objective Page - Objective 1: Generate Embeddings	96
39	TGD Dashboard Objective Page - Objective 2: Compute Cosine Similarities	96
40	TGD Dashboard Objective Page - Objective 3: Form a Feature Vector	97
41	TGD Dashboard Objective Page - Objective 4: Train History	98
42	TGD Dashboard Objective Page - Objective 5: Evaluate	99

Equation		Page
1	TF-IDF	16
2	TF-IDF Vector	19
3	GloVe Embeddings	23
4	DeBERTa Embeddings	26
5	Training Dataset	30
6	Ridge Prediction Function	30
7	Ridge Loss Function	30
8	Cosine Similarity	31
9	PCC	34
10	MSE	35
11	MAE	36
12	Feature Vector	42

CHAPTER I

INTRODUCTION OF THE STUDY

This presents the A Weighted Ensemble Approach to Semantic Similarity Using TGD. This chapter mainly consists of the Background of the Study, Objectives of the Study, Conceptual Framework, Definition of Terms, Significance of the Study, and Scope and Limitation of the Study.

A. Background of the Study

In the field of natural language processing (NLP), effectively representing textual data is crucial for the success of various applications, including text classification, information retrieval, and sentiment analysis. The choice of representation has been shown to significantly impact a model's ability to understand and process human language [1], [2], [3]. Among the earliest and most widely adopted methods, Term Frequency-Inverse Document Frequency (TF-IDF) has gained prominence for its simplicity and computational efficiency in transforming textual information into numerical form. TF-IDF is a statistical method that identifies important words in a document by considering how frequently they appear in a specific document compared to their occurrence across all documents [4]. Despite its usefulness, TF-IDF has notable limitations. It does not capture the relationships between words and cannot handle different meanings of the same word (polysemy) or similar meanings of different words (synonymy) [4], [5], [6]. However, its term weighting mechanism allows for effective differentiation of important words in a corpus, giving more emphasis to informative or distinguishing terms [7], [8]. Additionally, the term frequency component helps highlight

repeated occurrences of key words, which can be beneficial in tasks where word repetition indicates relevance [9]. TF-IDF also performs well on datasets with varied term distributions, providing a simple but reliable method for identifying significant terms even in the presence of rare words [10]. To address some of these limitations, word embedding techniques such as Global Vectors for Word Representation (GloVe) were developed. These methods convert words into dense vectors that reflect their semantic and syntactic properties by learning patterns from large text corpora [11]. However, word embeddings like GloVe are static. They generate only one vector per word regardless of the context in which it appears, making it difficult to distinguish between different meanings of the same word [12]. Building on this idea, contextual embedding models such as DeBERTa generate word representations that vary depending on surrounding words, allowing the same word to have different vectors based on its usage within a sentence [13]. However, DeBERTa also has limitations. While it relies on rich contextual cues to generate effective representations, it may fail to fully capture sentence-level semantic similarity and can produce highly anisotropic embedding spaces, which further constrain its performance, especially without fine-tuning, in datasets where contextual variation is limited [13], [14], [15].

Given these limitations, researchers propose augmenting document representations with semantic features [16]. While TF-IDF offers computational efficiency and simplicity, it lacks the capacity to capture deeper semantic relationships [4]. GloVe effectively encodes general word semantics but is limited by its inability to account for contextual variation [12]. DeBERTa, in contrast, is designed to model contextual dependencies. However, its effectiveness is highly dependent on the presence

of rich contextual information, and it can produce anisotropic embedding spaces [13], [14], [15]. Incorporating semantic information, such as word embeddings and contextual embeddings, could enhance all models' ability to capture nuanced semantic relationships between documents or sentences. Integrating these methods can help address their individual weaknesses and lead to a more effective and balanced models for text representation [16].

B. Objectives of the Study

The general objective of this study is to improve semantic similarity prediction by combining cosine similarity scores from TF-IDF, GloVe, and DeBERTa embeddings using a regression-based ensemble model.

Specifically, the study aimed to:

1. generate TF-IDF scores, GloVe embeddings, and DeBERTa embeddings to capture term importance, global semantic meaning, and deep contextual information from the text;
2. compute cosine similarity scores for each embedding method;
3. form a feature vector from the three similarity scores;
4. train a Ridge Regression model to learn the optimal weighted combination of the similarity scores for predicting semantic similarity; and
5. evaluate the ensemble model against standalone TF-IDF, GloVe, and DeBERTa models using MSE, MAE, Pearson Correlation, Accuracy, and F1-score.

C. Conceptual Framework

This study is grounded in the concept (see Figure 1) of enhancing semantic text representation by combining the cosine similarities derived from TF-IDF, GloVe, and DeBERTa embeddings. This approach leverages both statistical and contextual representations to improve the model’s ability to capture meaning, including context, synonymy, and polysemy. The framework follows a machine learning pipeline machine:

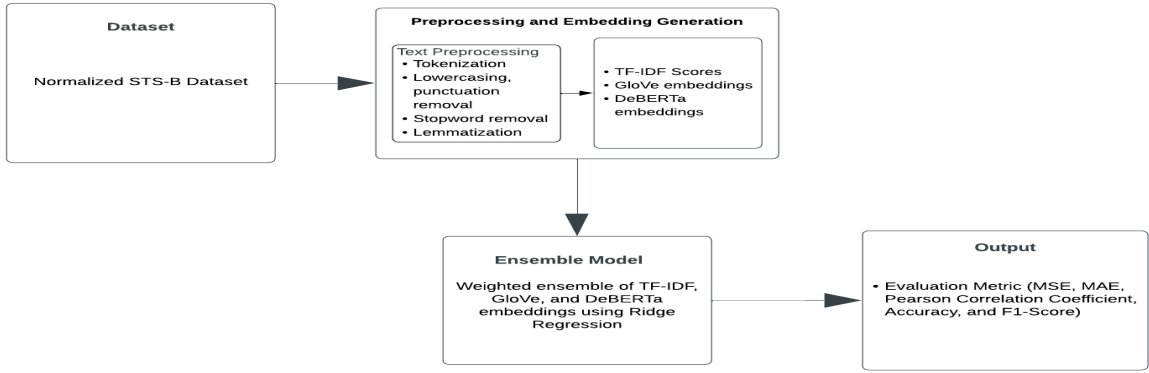


Figure 1 Conceptual Framework

The conceptual framework in Figure 1 presents a weighted ensemble model that improves semantic similarity prediction by combining multiple text representations. The process begins with the *Normalized STS-B Dataset*, where sentence pairs are preprocessed through tokenization, stopwords removal, and lemmatization. Embedding generation follows, producing TF-IDF scores to reflect term importance, GloVe embeddings to capture global semantic meaning, and DeBERTa embeddings to retain contextual understanding. Cosine similarity is computed for each embedding, resulting in a feature vector for every sentence pair. These features are fed into a Ridge Regression model, which learns the optimal contribution of each similarity score. Model performance is evaluated using the Pearson Correlation Coefficient to measure the

strength of the linear relationship between predicted and actual similarity scores, while Mean Squared Error (MSE) and Mean Absolute Error (MAE) quantify the degree of prediction errors, and Accuracy and F1 Score assess the model's ability to correctly identify similar and dissimilar sentence pairs, providing a comprehensive view of semantic similarity effectiveness.

D. Definition of Terms

This section provides conceptual and operational key terms utilized in the study.

Accuracy. Gives the projected instances' percentage of total instances [55].

In this study, accuracy is used to evaluate the overall correctness of the hybrid model in predicting the appropriate class labels for given sentence pairs.

Bias. A systematic tendency of a model to produce unfair predictions due to inherent characteristics of the data or learning process [40].

In this study, bias is addressed using Ridge Regression, which applies L2 regularization to reduce the influence of overly dominant features and produce more stable predictions.

Contextual Embeddings. Word representations that capture the meaning of words based on their surrounding context in a sentence or document [12].

In the study, contextual embeddings from DeBERTa enhance the semantic richness of word representations by incorporating the influence of surrounding words.

Cosine Similarity. A metric that measures the cosine of the angle between non-two vectors, used to determine the similarity between two documents based on their vector representation [17].

In this study, cosine similarity is utilized to measure the semantic similarity between sentence vectors generated from different embedding methods.

DeBERTa (Decoding-enhanced BERT with Disentangled Attention). An advanced transformer model that captures rich contextual information through its disentangled attention mechanism [13].

In the study, DeBERTa provides context-aware word embeddings to improve semantic understanding of text data.

Deep Contextual Information. Encodes how word meaning changes depending on surrounding words and their positional relationships in a sentence [13].

In this study, deep contextual information refers to context-aware representations produced by DeBERTa that capture both word meaning and relative positional dependencies through disentangled self-attention.

Dimension. A component of the vector space that represents a specific lexicalized subtree or feature of a document [55].

In this study, a dimension refers to a single component of a vector representing a sentence in the embedding space, where each dimension encodes a specific semantic feature learned by GloVe or DeBERTa to capture meaning within a sentence pair.

Epoch. One complete pass of the entire training dataset through the model [36].

In this study, an epoch refers to one complete pass of the entire training dataset through the model during training.

F1 Score. A cumulative component that is tested to determine the overall influence of recall and precision to determine the overall impact of false negative and false positive instances across the entire accuracy [55].

In the study, F1 Score is used to evaluate the performance of the ensemble model in distinguishing semantically similar and dissimilar sentence pairs.

Feature Vector. A list of numbers that represents the important traits of a data item for machine learning.

In the study, a feature vector is a set of numerical values representing the similarity scores from TF-IDF, GloVe, and DeBERTa embeddings for a pair of sentences, used as input to the ensemble model to predict semantic similarity.

Global Vectors for Word Representation(GloVe). A word embedding technique that captures global semantic relationships based on word co-occurrence statistics [26].

In the study, GloVe embeddings contribute to capturing the global relationships between words in the corpus.

Global Semantic. The meaning of a word is captured from co-occurrence patterns across the entire corpus, reflecting how it relates to all other words in the dataset.

In this study, global semantics refers to the meaning of words derived from overall corpus statistics, modeled using GloVe to create numerical representations.

Gold Score. The human-annotated score provided in the Semantic Textual Similarity Benchmark (STS-B) dataset indicates the semantic similarity between two sentences.

In this study, the Gold Score serves as the ground truth used to evaluate the predicted similarity scores generated by the models.

Mean Absolute Error (MAE). A metric used to evaluate the accuracy of a regression model by calculating the average of the absolute differences between predicted and actual values [54].

In the study, MAE measures the average magnitude of errors without considering their direction, providing a straightforward indication of how close the predicted similarity scores are to the human-annotated scores.

Mean Squared Error (MSE). A metric used to evaluate the accuracy of a regression model by measuring the average of the squared differences between predicted and actual values [52].

In the study, MSE quantifies the average squared difference between the predicted similarity scores and the actual annotated scores.

Natural Language Processing. A field of artificial intelligence focused on the interaction between computers and human language, enabling machines to understand, interpret, and generate human language in a meaningful way.

In the study, NLP techniques are employed to preprocess, represent, and analyze text data for improved semantic similarity.

Patience. Early stopping is a training method that stops the model from continuing once its performance on a validation set stops improving [55].

In this study, patience refers to the number of consecutive training epochs without improvement in the model's performance on the validation set before training is stopped using early stopping.

Pearson Correlation Coefficient. A statistical measure that evaluates the strength and direction of the linear relationship between two continuous variables [51].

In the study, the Pearson Correlation Coefficient is used as a metric to assess how closely the predicted similarity scores align with human-annotated scores.

Ridge Regression. A linear regression model that includes a regularization term to prevent overfitting [40].

In the study, Ridge Regression combines TF-IDF, GloVe, and DeBERTa similarity features with L2 regularization to prevent overfitting and ensure stable predictions.

Semantic. The interpretation of the meaning of words, phrases, or texts, focusing on the relationships and meanings conveyed beyond mere word matching.

In the study, the hybrid model aims to enhance semantic text representation by integrating statistical importance and contextual understanding for more accurate text analysis.

Sentence Pair. Two sentences are evaluated together to determine the degree of semantic similarity between them [48].

In this study, a sentence pair refers to two sentences from the STS-B dataset that are compared to measure their semantic similarity.

Term Importance. The measure of how significant or informative a word is within a document or corpus, often expressed to capture its contribution to the meaning or content of the text [17].

In the study, term importance refers to the significance of a word in a text, measured by its TF-IDF value, indicating how well it represents the content of the text.

TF-IDF (Term Frequency-Inverse Document Frequency). A numerical statistic that shows the relevance of keywords to specific documents [4].

In the study, TF-IDF is used to compute weights for significant terms in the document corpus, highlighting important words.

TGD (TF-IDF, GloVe, and DeBERTa Embedding). A hybrid text similarity approach that combines statistical term importance (TF-IDF), global semantic relationships (GloVe), and deep contextual understanding (DeBERTa) by computing their respective cosine similarities.

In the study, TGD serves as the core representation mechanism, with these similarity scores combined through a weighted ensemble model using Ridge Regression to enhance the prediction of semantic similarity between sentence pairs.

Word Embeddings. Techniques in natural language processing that map words or phrases to dense vectors of real numbers that capture both the semantic and syntactic characteristics of words by learning from large text corpora [11].

In the study, word embeddings generated by GloVe provide both the global semantic meaning of terms in the corpus.

Vectors. A numeric representation of a document in term space [5].

In this study, a vector refers to the numeric representation of a document, where each dimension corresponds to a TF-IDF weight that reflects the importance of a specific term in that document.

E. Significance of the Study

The significance of this study is centered on the combination of TF-IDF, GloVe, and DeBERTa embeddings to improve semantic text representation, offering a balanced approach between traditional and modern methods in natural language processing.

Academic Institutions. The methodology used in this study aligns with current developments in natural language processing, offering academic institutions a reference for discussing hybrid approaches to text representation. It supports instruction by linking

foundational concepts such as TF-IDF with advanced neural embeddings, providing a balanced view of traditional and modern NLP techniques.

Researchers. The hybrid model combining TF-IDF, GloVe, and DeBERTa similarity scores offers a practical basis for exploring the impact of multiple embedding strategies in semantic similarity tasks. It provides a framework that can be further extended or evaluated in comparative studies involving other language models, regression strategies, or datasets.

Students. The study outlines the process of applying and evaluating multiple embedding methods, which students can use as a reference for academic projects, thesis work, or further study in machine learning, information retrieval, or computational linguistics.

F. Scope and Limitation of the Study

This study is focused on the A Weighted Ensemble Approach to Semantic Similarity Using TGD. The primary objective is to improve semantic similarity prediction by combining cosine similarity scores derived from TF-IDF, GloVe, and DeBERTa embeddings through a regression-based ensemble model.

The study uses an English-language dataset from publicly available sources and applies preprocessing techniques such as tokenization, stopword removal, and lemmatization. Cosine similarity is computed for each embedding type, and a Ridge Regression model is used to combine the resulting similarity scores. Performance is

evaluated using standard metrics, including Pearson Correlation Coefficient, MSE, MAE, Accuracy, and F1-score.

The scope of this study is limited to semantic similarity tasks such as sentence pair matching and sentence-level similarity. All experiments are carried out in a simulated environment using Python and open-source machine learning libraries. The study does not address other NLP tasks such as text classification, sentiment analysis, or question answering.

Among the limitations are constraints in labeled dataset availability and computational resources. In particular, the training and evaluation of transformer-based models like DeBERTa require significant GPU memory and processing power, which may limit the scale and speed of experimentation. The computational environment for this study was a Lenovo IdeaPad 3 15IAU7 laptop running Windows 11 with the following key specifications: 12th Gen Intel Core i7-1255U CPU (1.70 GHz) and 16.0 GB of installed RAM. As a result, some model configurations or larger datasets could not be explored due to hardware limitations.

The study does not include real-time or multilingual capabilities, potentially limiting its adaptability across different languages and dynamic environments. Model performance may vary based on text domain, dataset noise, or data quality and balance. Cultural or idiomatic nuances, which could affect semantic interpretation, are beyond the scope of this study.

Finally, the evaluation is restricted to select benchmark datasets and does not extend to user-based assessments or real-world deployments. The system is intended as a support tool to aid semantic analysis, not as a replacement for human interpretation.

CHAPTER II

REVIEW OF RELATED LITERATURE

Text representation plays a foundational role in Natural Language Processing (NLP) by influencing the performance of tasks such as semantic similarity, classification, and clustering. Traditional techniques like Term Frequency–Inverse Document Frequency (TF-IDF) provide strong statistical insight into word importance and remain effective for similarity tasks, though they have limitations in capturing deeper semantic relationships and contextual meaning [4], [9]. Word embedding methods such as GloVe were introduced to model global word co-occurrence patterns, offering an alternative representation of word semantics [26]. However, GloVe generates static vectors that cannot adapt to different contexts [12]. Transformer-based models like DeBERTa further attempt to incorporate contextual understanding through disentangled attention mechanisms that separate content and positional information [13]. While DeBERTa enhances contextual modeling, it may still struggle with anisotropy and multi-entity relationships without fine-tuning [14], [15]. Recognizing the strengths and weaknesses of each method, recent research has explored hybrid models that integrate multiple embeddings and apply ensemble techniques such as Ridge Regression in order to combine their outputs effectively [42], [43]. This chapter examines relevant literature on TF-IDF, GloVe, DeBERTa, and ensemble methods to establish the theoretical foundation for the hybrid semantic similarity approach adopted in this study

A. TF-IDF

Text similarity measurement is some way to measure the degree of semantic similarity between two texts, and it is a very important task for natural language processing. Text similarity measures have extremely widespread application in many fields, such as text duplicate detection field, image retrieval, information retrieval, the automatic generation of text areas, and text classification. TF-IDF method is a traditional statistics-based text similarity measure algorithm, which is constructed by text word frequency vector [17]. By considering the word or term frequency (TF) in the document as well as how unique or infrequent (IDF) a word is in the whole corpus, TF-IDF assigns higher values to topic representative words while devaluing the common words. The TF-IDF weighted value $w_{t,d}$ for the word t in the document d is thus defined as [17]:

$$W_{ij} = t f_{ij} \cdot idf \quad (1)$$

The weight of a word, as seen in (1), represents the importance of the j -th term in the i -th document and is calculated using the TF-IDF formula. In this context, $t f_{ij}$ refers to the term frequency, indicating how often the j -th term appears in the i -th document. The inverse document frequency (idf) component is given by $\log N/df$, where N represents the total number of documents in the corpus, and df is the number of documents containing the j -th term. This combination of term frequency and inverse document frequency ensures that terms common across many documents are down weighted, while terms that are more unique to individual documents receive higher weights.

Despite its simplicity and computational efficiency, TF-IDF struggles to capture deeper semantic relationships such as synonymy and polysemy and often overemphasizes rare terms without regard to contextual meaning [4], [5], [6]. While early sentiment

analysis models successfully used TF-IDF especially with traditional machine learning algorithms, its lack of contextual understanding limits performance on shorter or nuanced texts. In a study by Das [6], TF-IDF combined with a Random Forest classifier achieved 93.81% accuracy on IMDB and Amazon Alexa datasets but underperformed on context-sensitive inputs. Mutsaddi and Choudhary developed a weighted ensemble model that integrates TF-IDF vectors with BERT embeddings to enhance plagiarism detection in low-resource languages like Marathi. Their system captures both statistical and semantic features of text, improving the accuracy of paraphrase and plagiarism identification. The study demonstrated that this hybrid approach outperformed models that relied solely on either TF-IDF or BERT representations [18]. Another known issue of TF-IDF is vector sparsity, which impacts its generalizability. To address this, the Dense Cohort of Terms (dCoT) method was introduced to reconstruct missing information by mapping sparse TF-IDF vectors into dense representations. Although this improved classification accuracy, it still failed to capture contextual usage [7]. Similarly, Yulita et al. [19] used a TF-IDF cosine-similarity model in an essay e-testing system and reported a Mean Absolute Error (MAE) of 0.123, explaining that TF-IDF is highly sensitive to wording changes and often assigns different scores to sentences with similar meanings. This supports that TF-IDF focuses on word overlap and cannot fully capture semantic similarity.

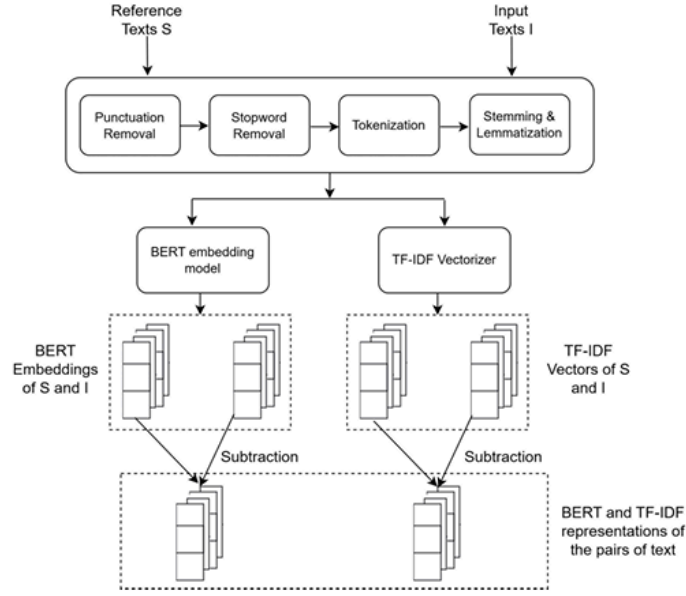


Figure 2 Pipeline for Extracting Features from Reference and Input Text Pairs [18]

Recent enhancements aim to align TF-IDF more closely with semantics. For instance, an extended model incorporating synonym mappings improved document similarity detection in Kazakh texts, as shown through Cosine, Dice, and Jaccard similarity scores [5]. In another study, researchers explored unsupervised sentence embedding models using TF-IDF. While improvements were noted in sentence similarity tasks, the method remained limited by TF-IDF’s static representation of words, which does not reflect word meaning changes across different texts. TF-IDF has also been used in academic text processing for tasks such as classification, clustering, and topic identification. It remains a practical tool in structured domains. Xiang [20] introduced an improved TF-IDF for literary text classification, while Hasugian et al. [21] used TF-IDF to organize and classify research abstracts based on content similarity. The underlying assumptions of TF-IDF are often questioned, especially the notion that a document’s word frequency accurately indicates its relevance. One study showed that frequent

repetitions of the same word (“word burstiness”) add little value to classification performance. To address such issues, Fan and Qin [22] introduced TF-IDCRF, incorporating class-based term relationships.

In the context of STS datasets, several studies have highlighted TF-IDF’s strong baseline performance. King et al. [23] reported a TF-IDF baseline achieving a Pearson correlation of 0.65271 across SemEval-2016 STS sub-tasks, demonstrating that lexical overlap provides a strong yet imperfect signal for semantic similarity. Similarly, Isbister and Sahlgren [24] showed that simple bag-of-words and TF-IDF representations perform remarkably well on STS-B-derived data, often rivaling more complex unsupervised semantic models. Additional evidence from Silva Junior et al. [25] and Arroyo-Fernández et al. [8] confirms that TF-IDF remains a competitive unsupervised baseline across STS domains.

Given a corpus $C = \{s_1, s_2, \dots\}$, each sentence s is represented using a TF-IDF (Term Frequency-Inverse Document Frequency) vector $\mathbf{v}_{\text{tfidf}}(s) \in \mathbb{R}^d$. This representation captures the importance of each word in the sentence relative to the entire corpus. The TF-IDF vector is computed as $\mathbf{v}_{\text{tfidf}}(s) = \text{TFIDF}(s, C)$ where each dimension corresponds to a weighted term frequency, adjusted by how unique the term is across the corpus. This allows the model to highlight informative words while down-weighting common ones, providing a lexical-level representation of the sentence.

$$\mathbf{v}_{\text{tfidf}}(s) = \text{TFIDF}(s, C) \quad (2)$$

The research and literature reviewed highlight the fundamental function of TF-IDF in statistical text representation. As shown in (2), each sentence is represented as a vector of weighted term frequencies adjusted by inverse document frequency, highlighting important words while down-weighting common ones [17]. This makes TF-IDF useful for tasks like document retrieval, text classification, clustering, and plagiarism detection [4], [5], [6], [18], [20], [21], [22]. TF-IDF is simple, interpretable, and computationally efficient. However, it cannot capture deeper semantic meaning, such as synonyms, polysemy, or context, and is sensitive to wording changes and sparse vectors [4], [5], [6], [7], [8], [19]. To address these limitations, TF-IDF is often combined with semantic embeddings like GloVe or DeBERTa, or enhanced with dense vector representations or class-based weighting [7], [18], [22]. In semantic textual similarity tasks, TF-IDF remains a strong unsupervised baseline, as highlighted by multiple studies [23], [24], [25]. In this study, TF-IDF is used as a complementary feature alongside GloVe and DeBERTa embeddings, providing lexical and frequency-based information that strengthens overall text representation.

B. GloVe

Understanding word semantics has evolved from basic frequency-based models to vector-based representations. One of the most impactful approaches in this space is GloVe (Global Vectors for Word Representation). The GloVe is a semantic vector space representation algorithm. It is representing documents based on word-word co-occurrences, and it is a count-based model. At the same time, the other word embeddings like word2vec is a predict based model. GloVe constructs a global word-context matrix and performs matrix factorization to generate word vectors that

encode meaning through both local context windows and global co-occurrence counts [26].

GloVe embeds words into a vector space where semantically similar words are geometrically close. This is achieved by transforming co-occurrence probabilities into ratios that reveal relationships between words. For instance, if "king" is often found in similar contexts as "queen," but also in distinct contexts from "man," GloVe can mathematically encode this relationship in vector space. The vectors thus preserve linear substructures in language, which enables tasks like word analogy solving (e.g., king - man + woman $\approx$ queen). This arithmetic property is a result of GloVe's formulation, which directly models ratios of co-occurrence probabilities, revealing patterns of similarity and difference among word meanings. The model efficiently constructs a co-occurrence matrix and uses a log-bilinear regression model to derive embeddings that exhibit meaningful substructures. It has shown competitive performance on analogy, similarity, and named entity recognition (NER) tasks, often outperforming Word2Vec and other matrix factorization approaches due to its global statistical grounding [26]. Word2Vec, introduced by Mikolov et al [27], utilizes shallow neural networks with two architectures: Continuous Bag of Words (CBOW) and Skip-gram, to predict words based on their context. This approach effectively captures complex semantic patterns and analogical relationships. However, Word2Vec treats words as indivisible tokens, which limits its ability to generate embeddings for out-of-vocabulary (OOV) words or capture morphological variations. FastText, developed by Facebook AI Research [28], extends Word2Vec by incorporating subword information, representing each word as a bag of character n-grams. This enables FastText to create embeddings for rare or unseen words

and better represent morphologically rich languages, though the inclusion of subword units may introduce noise, especially with irregular word forms. Several studies have empirically compared these embeddings. For instance, Tulu [29] evaluated pre-trained embedding models, Word2Vec, GloVe, and FastText, for semantic similarity tasks in Turkish. GloVe showed better accuracy and stability, though slightly less adaptive than FastText in handling rare or out-of-vocabulary terms. Complementing these findings, Malik et al. [30] examined the effectiveness of GloVe embeddings in capturing semantic relationships for predicting textual similarity and review helpfulness. Their results showed that GloVe achieved a Mean Absolute Error (MAE) of 0.2908 and Mean Squared Error (MSE) of 0.1280, indicating a fair ability to model semantic similarity but limited sensitivity to contextual nuances. Further studies on the STS-B dataset reports Pearson correlation scores for GloVe embeddings ranging from 0.51 [31] to 0.58 [32], providing further empirical evidence of its performance in modeling semantic similarity.

Compared to traditional word representations like TF-IDF, which fail to capture syntactic and semantic relationships, GloVe offers a more expressive low-dimensional embedding that improves performance in document clustering and other downstream tasks. Additionally, GloVe vectors are well-suited for visualization, distance-based similarity measures (e.g., cosine similarity), classification, or summarization tasks [33]. However, GloVe shares limitations inherent to static embedding models. Most notably, it fails to capture context-sensitive word meanings, treating polysemous words the same. Its static embedding nature prevents it from distinguishing between different senses of polysemous expressions, such as ‘Chicago,’ ‘Bulls,’ and the combined phrase ‘Chicago Bulls.’ Because it relies on global word-frequency statistics, it lacks the contextual

awareness needed to capture such distinctions, limiting its effectiveness in context-sensitive tasks [34].

A GloVe, as presented in (3), sentence embedding is computed by averaging the pre-trained word vectors of the words in the sentence. Given a set of GloVe embeddings $G : \text{word} \rightarrow \mathbb{R}^{300}$, the embedding for a sentence s is defined as $\mathbf{v}_{\text{glove}}(s) = \frac{1}{|T|} \sum_{w \in T} G(w)$, where $T = \{w \in s : w \in \text{vocab}(G)\}$. This method effectively represents the semantic meaning of a sentence by combining the vector representations of its known words.

$$\mathbf{v}_{\text{glove}}(s) = \frac{1}{|T|} \sum_{w \in T} G(w) \quad \text{where } T = \{w \in s : w \in \text{vocab}(G)\} \quad (3)$$

The research and literature reviewed highlights GloVe (Global Vectors for Word Representation) as an effective method for capturing semantic relationships through global word co-occurrence patterns. Unlike predictive models such as Word2Vec, GloVe constructs a co-occurrence matrix and uses a weighted least squares objective to generate word vectors that reflect both syntactic and semantic similarities [26]. A sentence embedding can be computed by averaging the vectors of individual words, as shown in (3), producing a fixed-length representation that summarizes the semantic content of the sentence. Compared to traditional lexical models like TF-IDF, GloVe provides richer low-dimensional embeddings that improve performance in tasks such as clustering, similarity detection, and classification [28], [29], [30], [33]. Studies report that GloVe embeddings achieve moderate to strong performance in semantic similarity tasks, including STS-B, further supporting their effectiveness [30], [31], [32]. However, as a static embedding model, GloVe cannot capture context-dependent meanings or

distinguish between different senses of polysemous words [29],[34]. In this study, GloVe is used to provide global semantic insight, complementing TF-IDF’s lexical information and DeBERTa’s contextual embeddings, thereby strengthening overall text representation.

C. DeBERTa

Transformer-based models have become foundational in modern Natural Language Processing (NLP), with BERT (Bidirectional Encoder Representations from Transformers) serving as a seminal breakthrough. Introduced by Devlin et al. [35], BERT utilizes a masked language modeling (MLM) objective and next sentence prediction (NSP) to learn deep bidirectional representations by conditioning on both left and right context. This enables BERT to generate rich contextual embeddings, outperforming earlier models on tasks like question answering and natural language inference. Its embeddings have also been effectively integrated into hybrid systems. For example, Mutsaddi and Choudhary [18] combined TF-IDF with BERT embeddings to enhance plagiarism detection in low-resource languages, demonstrating improved accuracy through the fusion of statistical and semantic features.

Despite its strengths, BERT’s reliance on the NSP objective has been considered a limitation, as studies suggest it offers limited benefit and may hinder certain downstream tasks. In response, Liu et al. [36] introduced RoBERTa (Robustly Optimized BERT Pretraining Approach), which eliminated the NSP task, expanded training data, and applied dynamic masking strategies. These modifications led to improved performance across several NLP benchmarks. Nonetheless, RoBERTa maintained the same approach

of jointly encoding content and positional information, which can limit its effectiveness in modeling complex linguistic structures.

An advancement addressing this issue is DeBERTa (Decoding-enhanced BERT with Disentangled Attention), introduced by He et al. [13]. This model separates word content from positional information using a disentangled attention mechanism. By representing each token with two vectors - one for content and one for position - DeBERTa more accurately models syntactic and semantic relationships. It also enhances the MLM task by incorporating absolute positional embeddings during the decoding stage, enabling the model to distinguish between words with similar contexts but different grammatical roles.

The DeBERTa-V3-Base model was selected as the contextual embedding component due to its optimal trade-off between semantic representation performance and computational efficiency. Among the available variants in the DeBERTa-V3 family: namely XSmall, Small, Base, and Large. The Base model comprises 12 transformer layers, a hidden size of 768, and approximately 86 million parameters, making it a scalable and resource-efficient choice suitable for standard research environments [13]. While larger variants such as DeBERTa-V3-Large offer marginal performance improvements, they demand substantially greater computational resources, making the Base variant more practical for integration into hybrid architectures. DeBERTa-V3-Base retains all the architectural advancements of the V3 series, including the disentangled attention mechanism, which separates content and position embeddings to enhance syntactic and semantic modeling, and the improved SentencePiece tokenization with a 128K vocabulary. These innovations have demonstrated superior results in semantic

textual similarity (STS) tasks, with the Base model achieving competitive performance compared to larger and more resource-intensive alternatives [3], [37]. Nonetheless, some studies report that DeBERTa-V3-Base achieves limited performance on the STS-B dataset, with Pearson correlation scores of approximately 0.20–0.30 [38] and mean squared error/mean absolute error values of 0.363 and 0.455 [39], respectively. These findings suggest that, while DeBERTa-V3-Base demonstrates competitive performance, there are still areas where its effectiveness can be further improved.

Certain limitations have been observed in DeBERTa, which has been shown to produce anisotropic embedding spaces where representations occupy narrow regions and distances between samples are compressed. Although supervised fine-tuning improves its ability to distinguish between categories, alignment across the semantic space remains limited, and representations can be unevenly distributed [14], [15]. These characteristics suggest that additional adaptation or modeling strategies may be necessary to fully exploit the semantic information in DeBERTa embeddings, particularly for capturing nuanced semantic similarity.

A DeBERTa sentence embedding is obtained by passing the sentence s through a pre-trained transformer model D , which produces a sequence of hidden states $H = D(s) = [h_1, h_2, \dots, h_n]$, where each h_i is the contextual embedding of the i -th token. The final sentence embedding $\mathbf{v}_{\text{deberta}}(s) = \frac{1}{n} \sum_{i=1}^n h_i$ is computed by applying mean pooling over all token embeddings:

$$\mathbf{v}_{\text{deberta}}(s) = \frac{1}{n} \sum_{i=1}^n \mathbf{h}_i \quad \text{where } \mathbf{H} = D(s) = [\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_n] \quad (4)$$

The research and literature reviewed highlight DeBERTa as an advancement in contextual embedding models for NLP. Unlike BERT and RoBERTa, DeBERTa uses a disentangled attention mechanism that separates word content from positional information, improving its ability to capture syntactic and semantic relationships [13]. Absolute positional embeddings further enhance masked language modeling, allowing the model to distinguish words with similar contexts but different grammatical roles [13], producing richer contextual representations for tasks like semantic textual similarity. The study uses DeBERTa-V3 Base for its balance of semantic performance and computational efficiency [13], [37]. Sentence embeddings are obtained via mean pooling over token-level hidden states, as shown in (4), creating a unified vector representing overall sentence meaning. While DeBERTa captures deep contextual semantics, some studies report limitations in complex reasoning or multi-entity scenarios and moderate STS-B performance [14], [15], [38], [39]. When combined with TF-IDF and GloVe, DeBERTa provides complementary contextual information that contributes to sentence similarity prediction, enhancing the model’s ability to capture semantic nuances.

D. Ridge Regression

Ridge Regression is a linear regression technique with L2 regularization that helps control overfitting by penalizing large coefficient weights [40]. Prior studies suggest that very small L2 regularization strengths impose only mild constraints on model coefficients, whereas stronger regularization may suppress informative relationships and degrade performance [41]. This property makes ridge regression particularly suitable for semantic similarity tasks, where lexical overlap measures, static

embeddings, and contextual embeddings often produce highly correlated similarity signals.

Prior work has shown that raw contextualized embeddings exhibit geometric properties that limit their effectiveness when directly used with cosine similarity. Ethayarajh showed that contextualized representations are highly anisotropic and that occurrences of the same word across different contexts become increasingly dissimilar in higher layers, which leads to unstable cosine similarity behavior when used without task-specific calibration [42]. Similarly, Reimers and Gurevych found that sentence representations derived directly from transformer outputs using simple aggregation methods yield suboptimal performance on semantic similarity tasks and can underperform static embeddings such as GloVe when not fine-tuned or calibrated through downstream learning [43].

To address these limitations, several studies have emphasized the benefits of combining multiple similarity representations. Cer et al. demonstrated that hybrid transfer models integrating word-level and sentence-level representations consistently outperform single-representation approaches on semantic similarity benchmarks [44]. Aldarmaki and Diab further argued that sentence-level representations provide more robust semantic mappings than isolated word embeddings, as aggregation reduces sensitivity to context-specific noise and individual token-level inconsistencies [45]. In this context, ridge regression helps balance multiple similarity features by controlling coefficient values, improving the reliability of semantic textual similarity prediction.

Recent empirical studies have demonstrated that combining cosine similarity with pretrained embeddings and applying ridge regression can effectively model semantic

similarity in linguistic assessment tasks. Gaddipati et al. [46] showed that embeddings such as ELMo, BERT, GPT, and GPT-2, when combined with cosine similarity features between student and reference responses, produced scores closely aligned with human judgments when fed into isotonic, linear, and ridge regression models, with ridge regression delivering competitive and stable results. Similarly, Luhurkinanti et al. [47] validated the use of BERT-based contextual embeddings together with cosine similarity to model semantic similarity in Japanese Automatic Short Answer Grading (ASAG) tasks. Their findings indicated that even without extensive fine-tuning, cosine similarity scores between contextual embeddings reliably approximated manual grades, and the application of ridge regression further improved predictive accuracy by stabilizing feature contributions. Building on these findings, Huang et al. [40] addressed broader linguistic assessment challenges by proposing an ensemble framework that integrates multiple advanced NLP models, including DeBERTa, RoBERTa, BART, and T5, and combines their outputs through stacking with LightGBM and ridge regression to enhance the accuracy and reliability of evaluating semantic and syntactic aspects. In addition, the UNBNLP system at SemEval-2016 Task 1 applied ridge regression to combine features from various baseline and embedding-based STS approaches, achieving a Pearson correlation of 0.69940 and demonstrating that integrating multiple similarity measures can modestly improve STS performance [23].

Together, these studies highlight the benefit of combining pretrained embeddings, cosine similarity measures, and ridge regression for reliable semantic similarity estimation, an approach adopted in this study. Specifically, this study uses a weighted ensemble model where similarity scores from TF-IDF, GloVe, and DeBERTa embeddings

are combined into a feature vector for each sentence pair. This vector is then passed to a Ridge Regression model, which learns an optimal linear combination of the features to predict the semantic similarity label. The training dataset is denoted as:

$$\{(x_i, y_i)\}_{i=1}^N \quad (5)$$

where each x_i is a feature vector comprising the combined cosine similarity scores from TF-IDF, GloVe, and DeBERTa embeddings, and y_i is the corresponding human-annotated scores. The model is trained to map each input vector x_i to its target y_i .

The prediction function of Ridge Regression is presented in (6):

$$\hat{y} = w_{\text{tfidf}} \cdot s_{\text{tfidf}} + w_{\text{glove}} \cdot s_{\text{glove}} + w_{\text{deberta}} \cdot s_{\text{deberta}} + b \quad (6)$$

where $\hat{y}$ is the predicted similarity score, $\mathbf{w}$ is the learned weight vector, and b is the bias term. This linear model attempts to approximate the target y_i based on the input features x_i .

The parameters $\mathbf{w}$ and b are learned by minimizing the following loss function:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \alpha \|\mathbf{w}\|^2 \quad (7)$$

This loss function combines the mean squared error between predicted and true similarity scores with an L_2 regularization term. The regularization is controlled by the non-negative parameter α , which discourages large weights and helps prevent overfitting, improving the model's generalization to unseen sentence pairs.

The research and literature reviewed highlight Ridge Regression as a regularized linear model that mitigates overfitting and improves generalization by controlling coefficient values [40], [41]. Studies demonstrate that combining cosine similarity features from pretrained embeddings with Ridge Regression enhances predictive performance in semantic similarity tasks [42], [43], [44], [45], [46], [47]. Building on these insights, this study employs a weighted ensemble approach, where cosine similarity scores derived from TF-IDF, GloVe, and DeBERTa are combined into a feature vector and passed through a Ridge Regression model to estimate semantic similarity between sentence pairs.

E. Cosine Similarity

Cosine similarity is used to evaluate the semantic closeness between sentence pairs based on different embedding methods. Each sentence is independently encoded using three representations: TF-IDF, GloVe, and DeBERTa. For each embedding method, cosine similarity is computed between the vector representations of the two sentences, producing three separate similarity scores per sentence pair. Cosine similarity is used to evaluate the semantic closeness between sentence pairs in the dataset. This formula is used to compute a similarity score between each sentence pair in the dataset [17].

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (8)$$

A and B represent the sentence vectors generated. The resulting score reflects how semantically similar the two sentences are; a value closer to 1 indicates strong similarity, while a value closer to 0 indicates dissimilarity [17].

In this study, we use the Semantic Textual Similarity Benchmark (STS-B) [48] dataset variant where all gold similarity scores are normalized between 0 and 1, as this range is more consistent with the cosine similarity measurement [49], which also outputs values between 0 and 1. This normalization ensures that the model’s predicted similarity scores are directly comparable to the cosine-based scales [50], facilitating a more accurate and interpretable evaluation of semantic relationships.

Table I Sample sentence pairs with cosine similarities and actual scores

No.	Sentence Pair	Cosine Similarity	Actual Score
1	sentence 1: A woman is putting on a sun glasses. sentence 2: A woman puts on sunglasses.	TF-IDF: 0.091 (Dissimilar) GloVe: 0.543 (Similar) DeBERTa: 0.981 (Similar)	1
2	sentence 1: A man is eating food. sentence 2: A man is eating something.	TF-IDF: 0.542 (Similar) GloVe: 0.861 (Similar) DeBERTa: 0.978 (Similar)	0.84
3	sentence 1: A toddler walks down a hallway. sentence 2: A little girl is walking down a hallway.	TF-IDF: 0.407 (Dissimilar) GloVe: 0.786 (Similar) DeBERTa: 0.982 (Similar)	0.72
4	sentence 1: A plane rides on a road. sentence 2: An airplane moves along a runway.	TF-IDF: 0.000 (Dissimilar) GloVe: 0.699 (Similar) DeBERTa: 0.892 (Similar)	0.56
5	sentence 1: A man is running. sentence 2: A man is singing.	TF-IDF: 0.235 (Dissimilar) GloVe: 0.926 (Similar) DeBERTa: 0.987 (Similar)	0.25
6	sentence 1: The man is riding a horse. sentence 2: A woman is using a hoe.	TF-IDF: 0.000 (Dissimilar) GloVe: 0.461 (Dissimilar) DeBERTa: 0.979 (Similar)	0

Table I presents cosine similarity scores computed separately using TF-IDF, GloVe, and DeBERTa embeddings for selected sentence pairs. To interpret these scores, this study adopts an evidence-based threshold supported by recent findings. The study of Kriuk and Kriuk [51] showed that both cosine similarity and Pearson correlation achieve optimal performance on the STS-B dataset at approximately the 0.5 threshold range,

identifying it as a stable cutoff for separating similar and dissimilar sentence pairs across accuracy and F1 evaluations. Following this finding, the computed similarity scores in this study are binarized using 0.5 as the decision threshold, where scores ≥ 0.5 are labeled as similar and scores < 0.5 are labeled as dissimilar. This binarization procedure is also consistent with prior work by Theron [52], who converted STS Benchmark similarity annotations into binary labels based on the SemEval semantic similarity rubric, which describes sentences as semantically similar when they are roughly equivalent although some information may differ.

The results in Table I demonstrate how TF-IDF, GloVe, and DeBERTa differ in their ability to capture semantic similarity between sentence pairs. TF-IDF assigns higher importance to words that appear frequently within a sentence but infrequently across the corpus, making it particularly sensitive to shared vocabulary. As a result, it performs well when sentences have exact or near-exact wording. For example, TF-IDF effectively distinguishes unrelated sentences such as sentence pair 6, “The man is riding a horse” and “A woman is using a hoe,” due to their complete lack of overlapping terms. It also detects distinctions between actions like sentence pair 5, “A man is running” and “A man is singing,” recognizing that even though the sentence structure is similar, the key verbs differ, and those differences are not masked by frequent common terms. However, this same reliance on exact word matches reveals TF-IDF’s limitations when synonymy or paraphrasing is involved. It struggles to recognize semantic similarity when the meaning is preserved but expressed using different vocabulary. For example, in sentence pair 3, “A toddler walks down a hallway” and “A little girl is walking down a hallway,” TF-IDF fails to capture the equivalence between “toddler” and “little girl.”

In contrast, GloVe and DeBERTa offer complementary strengths that address these limitations. GloVe captures global semantic relationships by learning from word co-occurrence patterns across large corpora. This allows it to identify that words like “toddler” and “girl” or “sunglasses” and “sun glasses” (as seen in sentence pair 1) often appear in similar contexts, thus enabling better recognition of paraphrased expressions. DeBERTa, on the other hand, excels in capturing contextual meaning at the sentence level by leveraging transformer-based attention mechanisms. It can interpret how words interact based on their surrounding context, making it particularly effective in understanding syntactic variation and deeper nuances in meaning. However, both GloVe and DeBERTa may overgeneralize, treating thematically or syntactically similar pairs as equivalent even when the underlying meanings differ. For instance, they may incorrectly judge sentence pair 5, “A man is running” and “A man is singing,” as similar, despite the clear distinction in actions. These findings underscore the importance of combining different models that focus on exact word matching with those that capture deeper meaning to improve the measurement of sentence similarity

F. Metrics

To evaluate how closely the predicted similarity scores align with the human-annotated scores, the Pearson Correlation Coefficient (PCC) can be employed [48].

$$r = \frac{\sum_{i=1}^n (g_i - \bar{g})(t_i - \bar{t})}{\sqrt{\sum_{i=1}^n (g_i - \bar{g})^2} \cdot \sqrt{\sum_{i=1}^n (t_i - \bar{t})^2}} \quad (9)$$

where n is the number of text pairs, g_i , t_i are the i^{th} score of the gold standard and test pairs scores, respectively. The $\bar{g}$ and $\bar{t}$ are the mean of the ground truth and test scores

[53]. Pearson correlation coefficient quantifies linear relationships and ranges from -1 to $+1$, where values closer to ± 1 indicate stronger associations. To interpret the strength of the correlation, this study follows the guidelines of Schober et al. [54], who define $r \geq 0.70$ as a strong correlation, $0.40\text{--}0.69$ as moderate, and below 0.40 as weak. These thresholds provide clearer context for evaluating how well the model's predictions align with human-annotated scores in the STS-B dataset.

Table II A conventional approach to interpreting a correlation coefficient [54]

Absolute Magnitude of the Observed Correlation Coefficient	Interpretation
0.00–0.10	Negligible correlation
0.10–0.39	Weak correlation
0.40–0.69	Moderate correlation
0.70–0.89	Strong correlation
0.90–1.00	Very strong correlation

Additionally, to evaluate how closely the predicted similarity scores align with the human-annotated scores, Mean Squared Error (MSE) can be employed [55].

$$\text{mean squared error (mse)} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (10) [56]$$

where y_i represents the ground-truth similarity score for the i -th sentence pair from the dataset, and $\hat{y}_i$ is the predicted cosine similarity computed using (8). The MSE calculates the average squared difference between these values across all N sentence pairs. It is used to evaluate how close the model's predicted similarity scores are to the human-annotated scores with lower values indicating better semantic prediction accuracy.

The Mean Absolute Error (MAE) can also be employed as a complementary metric:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|. \quad (11)$$

Mean Absolute Error (MAE) is derived from the unaltered magnitude (absolute value) of each difference. Unlike MSE, which disproportionately penalizes larger deviations, MAE provides a more direct measure of the average error magnitude. Thus, lower MAE values indicate that the predicted similarity scores are, on average, closer to the human-annotated scores [57].

In order to quantitatively evaluate the performance of the proposed model, this study employs the confusion matrix and its associated evaluation metrics. In the context of semantic encoding, True Negative (TN) denotes the number of dissimilar text pairs correctly classified as negative, while True Positive (TP) represents the number of similar text pairs correctly identified as positive. False Negative (FN) refers to similar pairs incorrectly classified as dissimilar, and False Positive (FP) refers to dissimilar pairs incorrectly identified as similar [58]. These four values collectively describe the model’s prediction outcomes by illustrating its behavior across all evaluated cases, including correctly and incorrectly identified similar and dissimilar sentence pairs. The use of metrics such as F1-Score for evaluating semantic similarity tasks has also been supported in prior research, which demonstrated that cosine similarity can effectively distinguish between similar and dissimilar sentence pairs within a vector space representation [50].

The F1-score is the harmonic mean of precision and recall, capturing the balance between both metrics. A higher F1-score indicates stronger and more consistent model performance in identifying positive cases, as shown on Figure 3.

Accuracy measures the overall proportion of correctly classified instances, encompassing both similar and dissimilar sentence pairs, as shown on Figure 3.

The calculation formulas for these metrics are illustrated in Figure 3.

		Prediction	
		Similar	Dissimilar
Truth	Similar	True Positive (TP)	False Negative (FN)
	Dissimilar	False Positive (FP)	True Negative (TN)

Evaluation index

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Figure 3 Model performance evaluation indices and calculation formulas

CHAPTER III

METHODOLOGY

This chapter presents the methodological approach employed to develop and evaluate the proposed weighted ensemble model. The section outlines the research design, dataset description, data preprocessing techniques, model selection, training and validation procedures, evaluation metrics, and the technological tools utilized throughout the study. This structured methodology ensures the replicability of results and aligns with the objectives outlined in the earlier chapters.

A. Research Design

The study utilizes an experimental research design to combine TF-IDF with static word embeddings (GloVe) and contextual embeddings (DeBERTa). The focus is on evaluating the effectiveness of the hybrid model in semantic similarity tasks.

Dataset

The Semantic Textual Similarity Benchmark (STS-B) dataset serves as the basis for experimentation. This benchmark dataset includes sentence pairs in multiple languages, each accompanied by a human-annotated score indicating their degree of relatedness. For this study, the English subset is utilized to ensure linguistic consistency and focus.

Given that cosine similarity produces values between 0 and 1, this research utilized a variant of the STS-B dataset where similarity scores were already normalized to this same range. This approach enables a direct comparison between the model's predicted

similarity scores and the gold standard annotations, thereby supporting clearer interpretation and consistent evaluation.

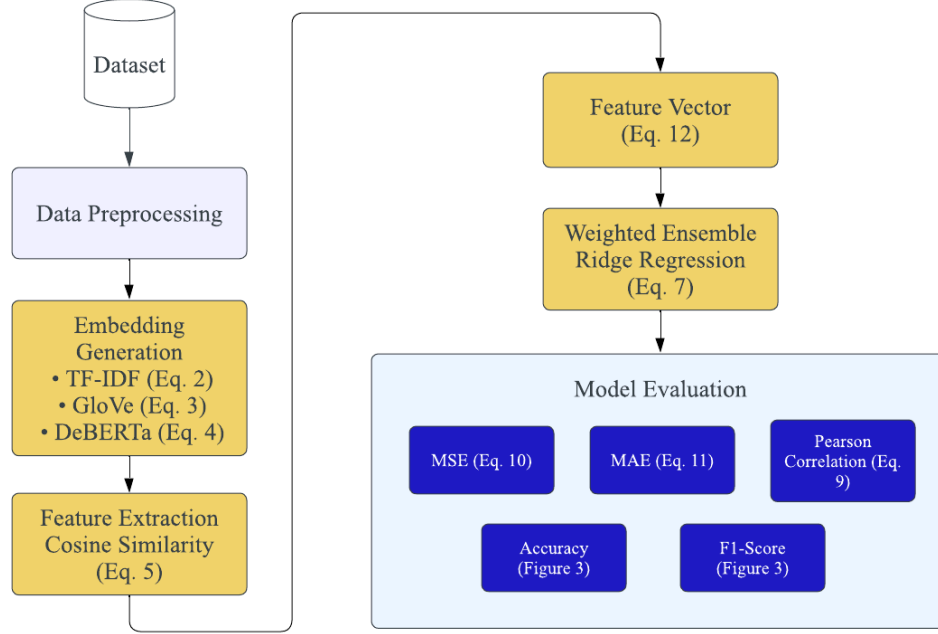


Figure 4 Research Design

The dataset is divided into training, development, and test splits, allowing systematic model training, hyperparameter tuning, and performance evaluation under controlled conditions. The sentence pairs encompass a wide range of semantic relations, from paraphrases and closely related statements to unrelated and contradictory sentences, providing a diverse foundation for evaluating sentence similarity models.

Data Preprocessing

To prepare the data for embedding generation, a tailored preprocessing pipeline is implemented. First, all text is standardized by converting it to lowercase and removing excess whitespace and punctuation. For the TF-IDF and GloVe embeddings, further processing is applied, including tokenization, stopwords removal, and lemmatization.

These steps ensure that the text is reduced to its most meaningful units, facilitating effective term weighting and vector mapping. In contrast, DeBERTa uses the original cleaned text without lemmatization to preserve syntactic and grammatical context, as this information is crucial for transformer-based embeddings. Each sentence is preprocessed accordingly, resulting in parallel inputs for the hybrid embedding generation step.

Embedding Generation

Three types of embeddings are used to represent sentence semantics. TF-IDF (Term Frequency–Inverse Document Frequency) captures term-level importance by weighing words based on their frequency in a sentence relative to their rarity in the entire corpus, as defined in (2) (TF-IDF Vectors, p. 19). GloVe (Global Vectors for Word Representation) generates dense word embeddings based on word co-occurrence statistics across a large corpus, with sentence-level embeddings formed by averaging word vectors, as shown in (3) (GloVe Embeddings, p. 23). DeBERTa (Decoding-enhanced BERT with disentangled attention) provides contextual embeddings by encoding each sentence using deep transformer layers and aggregating token-level hidden states through mean pooling, as expressed in (4) (DeBERTa Embeddings, p. 26). These embedding techniques collectively capture lexical frequency, global semantic relationships, and deep contextual understanding, providing a comprehensive representation of sentence meaning.

Feature Extraction

Figure 5 illustrates the process of extracting a feature vector for each sentence pair using three embedding methods: TF-IDF, GloVe, and DeBERTa. Each sentence is

represented separately through these models. Cosine similarity is computed between the two sentences for each embedding type, resulting in three similarity scores.

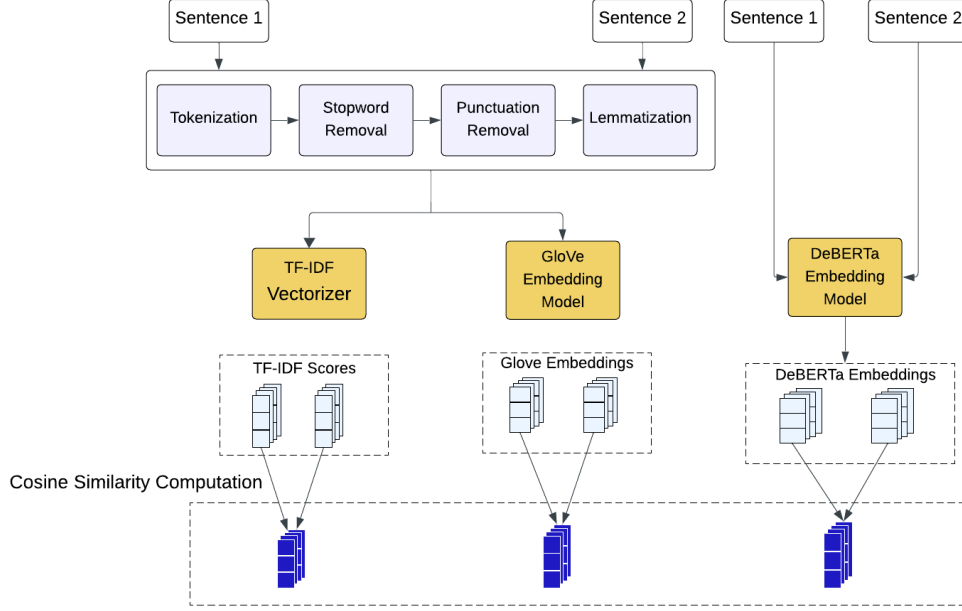


Figure 5 Feature Vector Extraction TF-IDF, GloVe, and DeBERTa Representation per Sentence Pair

Semantic similarity between sentence pairs is computed using features derived from three models: TF-IDF, GloVe, and DeBERTa. The similarity between sentence pairs is measured using cosine similarity, as defined in (5) (Training Dataset, p. 30), applied respectively to the TF-IDF vectors, GloVe embeddings, and DeBERTa embeddings to construct the final feature set.

Feature Vector

The final feature vector for each sentence pair is formed by combining the cosine similarity scores from the three models. Let $\mathbf{x} = [s_{\text{tfidf}}, s_{\text{glove}}, s_{\text{deberta}}]$, where each component represents the cosine similarity between the corresponding embeddings of the sentence pair. The first score, s_{tfidf} , compares the sentences based on word usage and

frequency. The second, s_{glove} , captures the general meaning of the words using pre-trained word vectors. The third score, s_{deberta} , measures deeper, context-based similarity using a transformer model. By combining these three scores together, the model can better understand and predict how similar two sentences really are.

$$\mathbf{x} = [s_{\text{tfidf}}, s_{\text{glove}}, s_{\text{deberta}}] \quad (12)$$

Weighted Ensemble

To combine multiple vector representations, this study employs a weighted ensemble approach using Ridge Regression, as previously detailed in Chapter 2 (Equations 5–7, p. 30-31). The final feature vector for each sentence pair consists of three cosine similarity scores, one each from TF-IDF, GloVe, and DeBERTa embeddings. These vectors are used as input to the Ridge Regression model, which learns the optimal linear combination of the features to approximate the annotated similarity score.

Model Evaluation

Model performance is evaluated using several evaluation metrics. The Pearson Correlation Coefficient (PCC) measures the linear relationship between predicted similarity scores and the human-annotated scores, where higher values indicate stronger alignment with human judgment. To quantify prediction errors, Mean Squared Error (MSE) and Mean Absolute Error (MAE) are computed. Accuracy and F1 Score are also used to evaluate the model’s ability to distinguish between similar and dissimilar sentence pairs.

B. Data Collection

This study utilizes the Semantic Textual Similarity Benchmark dataset, a publicly available resource designed for evaluating semantic similarity between sentence pairs.

Dataset Source

The Semantic Textual Similarity Benchmark (STS-B) dataset is obtained from the Hugging Face Datasets library. It originates from the original benchmark used in SemEval 2017 Task 1, which evaluates the degree of semantic equivalence between pairs of English sentences. This study employs the standard STS-B dataset variant, where similarity scores are represented within the 0 to 1 range. This scoring format supports clear interpretation of similarity levels and facilitates consistent comparison between model outputs and reference values. Furthermore, the dataset focuses solely on English sentence pairs, ensuring consistency in evaluating the performance of the models.

Size and Structure

The English STS-B dataset contains a total of 8,628 sentence pairs, each annotated with a real-valued similarity score. The dataset is organized into three predefined splits: 5,749 pairs for training (66.6%), 1,500 for development/validation (17.4%), and 1,379 for testing (16%). Each entry consists of two English sentences accompanied by a human-annotated score that represents their degree of semantic relatedness.

Features/Attributes

Table III Attributes of the STS-B dataset used in this study

Attribute	Description
sentence1	The first sentence in the pair.
sentence2	The second sentence in the pair.
similarity_score	A similarity score ranging from 0 to 1 in the normalized variant, representing the degree of semantic similarity between the two sentences.

Table IV Similarity scores with explanations and examples

Score	Sentence Pair	Explanation
1	1. The bird is bathing in the sink. 2. Birdie is washing itself in the water basin.	<i>The two sentences are completely equivalent, as they mean the same thing.</i>
0.8	1. Two boys on a couch are playing video games. 2. Two boys are playing a video game.	<i>The two sentences are mostly equivalent, but some unimportant details differ.</i>
0.6	1. John said he is considered a witness but not a suspect. 2. "He is not a suspect anymore." John said.	<i>The two sentences are roughly equivalent, but some important information differs/missing.</i>
0.4	1. They flew out of the nest in groups. 2. They flew into the nest together.	<i>The two sentences are not equivalent, but share some details.</i>
0.2	1. The woman is playing the violin. 2. The young lady enjoys listening to the guitar.	<i>The two sentences are not equivalent, but are on the same topic.</i>
0	1. The black dog is running through the snow. 2. A race car driver is driving his car through the mud.	<i>The two sentences are completely dissimilar.</i>

This dataset provides pairs of English sentences and a numeric similarity score representing their degree of semantic relatedness. Table IV summarizes the key attributes included in the dataset.

C. Data Preprocessing

To ensure consistency and improve model performance, the dataset undergoes preprocessing. The text preprocessing process begins by converting all text to lowercase to standardize the input across the embedding methods. Basic cleaning is applied to ensure the text remains grammatically intact, which includes removing any extra spaces between words and punctuation. For TF-IDF and GloVe, the text is tokenized, and common stop words are removed to eliminate low-information terms that could distort similarity measurements between documents. Additionally, lemmatization is applied to reduce inflected words to their base form, grouping semantically similar words together. For the DeBERTa model, tokenization is handled automatically within the model's pipeline, preserving the raw cleaned sentence without lemmatization to maintain the contextual information the model relies on.

D. Machine Learning Models

Tables V and VI present a comparison of the embedding models based on their key idea, dimensionality, strengths, and weaknesses. Table V includes static embedding models such as Word2Vec, GloVe, and FastText, which generate fixed word representations, while Table VI includes contextual models such as BERT, RoBERTa, and DeBERTa, which produce context-aware representations of words.

Table V Static word embeddings comparison table

Method	Key Idea	Dimensionality	Strengths	Weaknesses
Word2Vec (CBOW & Skip-gram) [29]	Predicts target from context (CBOW) or context from target (Skip-gram)	100-300	Efficient; captures syntactic & semantic relations; supports analogies	No OOV handling; no subword or morphological modeling
FastText [17]	Extends Word2Vec by incorporating subword (character n-gram) information		Handles rare/unknown words via subword info, robust to misspellings	Can introduce noise due to inclusion of irrelevant or irregular subword units
GloVe [26]	Factorizes a co-occurrence matrix	50-300	Global co-occurrence information, interpretable	Cannot handle OOV words; does not account for subword or morphological structure

Table VI Contextual embeddings comparison table

Model	Key Idea	Dimensionality	Strengths	Weaknesses
BERT [35]	Masked Language Modeling (MLM) + Next Sentence Prediction during pre-training	Base: 768 Large: 1024	Deep bidirectional transformer encoder, which allows it to pre-train representations that jointly condition on both left and right context in all layers.	The NSP objective may not be effective for certain tasks; pre-training is time-consuming and resource-intensive
RoBERTa [36]	Optimized BERT pretraining: dynamic masking, removal of Next Sentence Prediction, larger datasets, longer training	Base: 768 Large: 1024	Improves on BERT by training the model longer, with more data, removing the next sentence prediction objective, and dynamically changing the masking pattern applied to the training data.	Lacks of NSP means no modeling of inter-coherence, model still relies on the same architecture as BERT
DeBERTa [13]	Disentangled attention mechanism separating content and position information; enhanced decoding with an improved mask decoder	Base: 768 Large: 1024	Improves the attention mechanism by disentangling word content and position, enhancing the model's ability to capture word relationships.	Increased complexity due to added layers and attention modification

E. Model Training and Evaluation

The model is trained using the Normalized STS-B dataset, which contains 8,628 sentence pairs with human-annotated scores. The provided development (dev) set is used for validation during model development.

Evaluation on the test split uses the Pearson Correlation Coefficient (PCC) to examine how well the predicted scores follow the actual similarity scores. To capture prediction errors, Mean Squared Error (MSE) and Mean Absolute Error (MAE) are also computed, quantifying the average squared and absolute differences between predictions and the ground truth. Similarity scores are then converted into binary labels using a decision threshold. Based on this threshold, Accuracy and F1 Score are reported on the test split. Together, these metrics provide a balanced evaluation of the model's ability to generate similarity scores that reflect human judgment while minimizing error and effectively distinguishing between semantically similar and dissimilar sentence pairs.

F. Evaluation Metrics

The performance of the proposed ensemble model is evaluated using a variety of evaluation metrics. Cosine similarity is computed independently for each sentence pair using TF-IDF, GloVe, and DeBERTa embeddings to measure semantic closeness based on different representation strategies. These similarity scores are then combined and used as input to a Ridge Regression model that predicts overall semantic similarity. Mean Squared Error (MSE) and Mean Absolute Error (MAE) are computed to quantify the degree of prediction errors, while the Pearson Correlation Coefficient (PCC) is employed to evaluate the strength of the linear relationship between the predicted similarity scores

and the human-annotated scores. In addition, Accuracy and F1 Score are calculated after converting similarity scores into binary labels using a decision threshold, allowing the assessment of the model's ability to identify semantically similar and dissimilar sentence pairs. Together, these metrics provide a comprehensive assessment of the model's semantic representation quality and predictive performance.

G. Tools and Environments

To build, train, and evaluate the hybrid model for semantic understanding, the study utilized the following technologies:

Programming Language

- Python 3.11: Python was the primary programming language used due to its rich ecosystem of libraries for data science, machine learning, and natural language processing (NLP).

Libraries

- scikit-learn: Used for implementing machine learning algorithms, such as classification models, and for computing metrics like accuracy, precision, recall, and F1-score.
- pandas: Used for data manipulation and cleaning, especially for handling the sentence pairs and labels in the dataset.
- NumPy: Utilized for numerical computations, including handling array operations and vectorized calculations.

- nltk: A library for natural language processing, helpful for tasks like tokenization and text preprocessing.
- Hugging Face: Used to load the pre-trained Microsoft/deberta-v3-base model for generating contextual embeddings. It also provides access to the STS-B Multilingual dataset (English subset).
- Hugging Face Datasets: Provides access to the STS-Benchmark (STS-B) dataset, including the English subset used in this study.
- TensorFlow/Keras: Implemented the Ridge regression model with L2 regularization
- PyTorch: Backend framework used by Hugging Face Transformers to run the DeBERTa model efficiently on GPU/CPU.
- SciPy: Specifically, the pearsonr function was used to compute the Pearson Correlation Coefficient (PCC) for evaluating similarity predictions.
- tqdm: Used for progress tracking during embedding generation and model training loops.

Hardware/Cloud Tools

- Google Colab: Served as the development environment, providing access to high-performance GPUs (including both T4 and A100) for training large models such as DeBERTa and enabling faster embedding computations.

H. Ethical Considerations

This study exclusively uses the normalized variant of the STS Benchmark dataset, a publicly available resource obtained from the Hugging Face Datasets platform. The dataset is sourced from recognized academic projects and contains no personal, identifiable, or sensitive information. Our use of the dataset complies fully with ethical standards concerning data usage, privacy, and academic integrity. Proper citations and acknowledgments have been included to respect intellectual property rights.

CHAPTER IV

RESULTS AND DISCUSSION

This chapter presents and explains the study's results in line with the five specific objectives stated in Chapter I. Each section corresponds to a particular objective and shows both the findings and their interpretation. The study aims to build a weighted ensemble model that predicts the similarity between two sentences using three types of embeddings: TF-IDF, GloVe, and DeBERTa, combined through Ridge Regression.

A. Generate TF-IDF scores, GloVe embeddings, and DeBERTa embeddings to capture term importance, global semantic meaning, and deep contextual information from the text

The first objective is to create three types of sentence embeddings that capture lexical, semantic, and contextual meaning. The dataset from the Hugging Face repository sentence-transformers/stsb was used, comprising 8,628 sentence pairs, divided into training (66.6%), development (17.4%), and testing (16%) sets. Figure 6 shows five rows from each dataset split. The first objective is to create three types of sentence embeddings that capture lexical, semantic, and contextual meaning. The dataset from the Hugging Face repository sentence-transformers/stsb was used, comprising 8,628 sentence pairs, divided into training (66.6%), development (17.4%), and testing (16%) splits. The training split is the largest portion of the dataset and is used to allow the model to learn patterns of semantic similarity by adjusting the model parameters during the learning process. The development split (also called the validation split) is used to fine-tune model configurations, such as parameter adjustments and performance monitoring, to prevent

overfitting and ensure generalization. Meanwhile, the testing split serves as an unseen dataset used exclusively for the final evaluation of the model to measure its actual performance and predictive capability. Figures 6, 7, and 8 show five rows from each dataset split.

	Dataset Split	sentence1	sentence2	score
0	Training	A plane is taking off.	An air plane is taking off.	1.00
1		A man is playing a large flute.	A man is playing a flute.	0.76
2		A man is spreading shredded cheese on a pizza.	A man is spreading shredded cheese on an uncoo...	0.76
3		Three men are playing chess.	Two men are playing chess.	0.52
4		A man is playing the cello.	A man seated is playing the cello.	0.85

Figure 6 Sentence Pairs with Similarity Scores (STS-B Training Split)

	Dataset Split	sentence1	sentence2	score
0	Development	A man with a hard hat is dancing.	A man wearing a hard hat is dancing.	1.00
1		A young child is riding a horse.	A child is riding a horse.	0.95
2		A man is feeding a mouse to a snake.	The man is feeding a mouse to the snake.	1.00
3		A woman is playing the guitar.	A man is playing guitar.	0.48
4		A woman is playing the flute.	A man is playing a flute.	0.55

Figure 7 Sentence Pairs with Similarity Scores (STS-B Development Split)

	Dataset Split	sentence1	sentence2	score
0	Test	A girl is styling her hair.	A girl is brushing her hair.	0.50
1		A group of men play soccer on the beach.	A group of boys are playing soccer on the beach.	0.72
2		One woman is measuring another woman's ankle.	A woman measures another woman's ankle.	1.00
3		A man is cutting up a cucumber.	A man is slicing a cucumber.	0.84
4		A man is playing a harp.	A man is playing a keyboard.	0.30

Figure 8 Sentence Pairs with Similarity Scores (STS-B Test Split)

The STS-B dataset provides balanced examples that show both similar and dissimilar sentence pairs. The wide range of scores allows the model to learn fine-grained differences in meaning. All sentences were converted to lowercase, stripped of punctuation, and filtered for stop words. Lemmatization was applied to TF-IDF and

GloVe to keep only root word forms. DeBERTa inputs were kept in their original form to preserve context. Preprocessing ensures that irrelevant characters and redundant word forms are removed, which helps reduce noise and improves the quality of vector representations. Figure 9 presents sentence pairs before and after text cleaning and normalization, while the implementation code used for the preprocessing steps is provided in **Appendix A.1 (p. 86)**.

	Raw Sentence 1	Raw Sentence 2	Preprocessed Sentence 1	Preprocessed Sentence 2
0	A plane is taking off.	An air plane is taking off.	plane taking	air plane taking
1	A man is playing a large flute.	A man is playing a flute.	man playing large flute	man playing flute
2	A man is spreading shredded cheese on a pizza.	A man is spreading shredded cheese on an uncooked pizza.	man spreading shredded cheese pizza	man spreading shredded cheese uncooked pizza
3	Three men are playing chess.	Two men are playing chess.	three men playing chess	two men playing chess
4	A man is playing the cello.	A man seated is playing the cello.	man playing cello	man seated playing cello

Figure 9 Preprocessed Dataset Overview

TF-IDF Embedding Generation

```
# Initialize and train the TF-IDF vectorizer using the preprocessed training sentences.
tfidf_vectorizer = TfidfVectorizer()
tfidf_vectorizer.fit(s1_pre_tr + s2_pre_tr)

# Transform the first 10 test sentence pairs into TF-IDF vectors and extract the top weighted terms from each sentence.
feature_names = tfidf_vectorizer.get_feature_names_out()
table_rows = []

for i in range(10):
    v1 = tfidf_vectorizer.transform([s1_pre_te[i]])
    v2 = tfidf_vectorizer.transform([s2_pre_te[i]])
    sim = sim_tfidf_te[i]
    nz1, nz2 = v1.nonzero()[1], v2.nonzero()[1]

    weights1 = sorted([(feature_names[j], v1[0, j]) for j in nz1], key=lambda x: -x[1])[0:4]
    weights2 = sorted([(feature_names[j], v2[0, j]) for j in nz2], key=lambda x: -x[1])[0:4]

    vec1_str = ", ".join([f"{t} ({w:.4f})" for t, w in weights1]) if weights1 else "<no terms>"
    vec2_str = ", ".join([f"{t} ({w:.4f})" for t, w in weights2]) if weights2 else "<no terms>"
```

Figure 10 Code Snippet for Generating TF-IDF Vectors

The code snippet in Figure 10 illustrates how TF-IDF vectors are generated from the preprocessed sentences. It converts each sentence into a numerical representation that emphasizes important and discriminative terms based on their frequency and rarity across the dataset.

The Term Frequency–Inverse Document Frequency (TF-IDF) method was applied to transform each preprocessed sentence into a numerical vector representation. TF-IDF assigns higher weights to words that are frequent within a sentence but rare across the entire dataset, emphasizing their importance in distinguishing sentence meaning. The TF-IDF weight is defined in (2) (TF-IDF Vectors, p. 19).

	Sentence 1	Top TF-IDF Terms (Sentence 1)	Sentence 2	Top TF-IDF Terms (Sentence 2)	Cosine Similarity
0	girl styling hair	styling (0.7317), hair (0.5652), girl (0.3809)	girl brushing hair	brushing (0.7059), hair (0.5874), girl (0.3959)	0.4828
1	group men play soccer beach	soccer (0.5118), beach (0.4549), play (0.4378), men (0.4217)	group boy playing soccer beach	soccer (0.5465), beach (0.4857), group (0.4294), boy (0.4047)	0.6733
2	one woman measuring another woman 's ankle	woman (0.6687), another (0.5612), one (0.4877)	woman measure another woman 's ankle	measure (0.6133), woman (0.6050), another (0.5077)	0.6895
3	man cutting cucumber	cucumber (0.7598), cutting (0.5630), man (0.3250)	man slicing cucumber	cucumber (0.7725), slicing (0.5423), man (0.3304)	0.6943
4	man playing harp	harp (0.8411), playing (0.4281), man (0.3306)	man playing keyboard	keyboard (0.7950), playing (0.4800), man (0.3708)	0.3281

Figure 11 TF-IDF Vectors of the First 10 Sentence Pairs from the Test Split Shown in Figure 8

Note: Only the top weighted non-zero terms for each sentence are shown for readability; full vectors contain thousands of dimensions.

The generated TF-IDF vectors clearly illustrate how words with higher distinctive value receive greater weights in their respective sentences. As shown in Figure 11, terms such as “styling,” “brushing,” and “cucumber” obtained high TF-IDF scores because they frequently appear within the sentence but are relatively uncommon across the overall

dataset. This weighting mechanism enables the representation to emphasize the most informative words that convey the core meaning of each sentence, as detailed in the study of Lan [17].

The variation in weights among identical words across sentences also indicates the model's sensitivity to word frequency. This means that TF-IDF effectively differentiates sentence meaning based on unique word usage [17]. Overall, the generated embeddings demonstrate the method's strength in highlighting lexical significance and filtering out less meaningful terms, resulting in cleaner and more focused sentence representations suitable for similarity computation.

Generation of GloVe Embeddings

```
# Load pre-trained GloVe embeddings and generate sentence embeddings by averaging
word vectors.
GLOVE_PATH = "/content/drive/MyDrive/glove.6B.300d.txt"

def load_glove(path):
    embeddings = {}
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            parts = line.rstrip().split(" ")
            embeddings[parts[0]] = np.asarray(parts[1:], dtype=np.float32)
    return embeddings, len(next(iter(embeddings.values())))

glove_dict, glove_dim = load_glove(GLOVE_PATH)

def glove_embed_sentence(text: str, dim=300):
    vecs = [glove_dict[t] for t in text.split() if t in glove_dict]
    return np.mean(vecs, axis=0).astype(np.float32) if vecs else np.zeros(dim,
dtype=np.float32)

# Generate GloVe sentence embeddings and cosine similarity for the first 10 test
sentence pairs.
table_rows_glove = []

for i in range(10):
    s1, s2 = s1_pre_te[i], s2_pre_te[i]

    v1 = glove_embed_sentence(s1, glove_dim)
    v2 = glove_embed_sentence(s2, glove_dim)

    sim = cosine_similarity([v1], [v2])[0, 0]
    vec1_str = ", ".join([f"{x:.4f}" for x in v1[:10]])
    vec2_str = ", ".join([f"{x:.4f}" for x in v2[:10]])
```

Figure 12 Code Snippet for Generating GloVe Embeddings

The code snippet in Figure 12 illustrates the process of generating sentence-level GloVe embeddings by averaging pre-trained word vectors. The resulting dense vectors capture the overall semantic meaning of each sentence based on global word co-occurrence statistics.

To complement the lexical representation of TF-IDF, this study employed Global Vectors for Word Representation (GloVe) to capture the broader semantic relationships between words. GloVe represents each word as a dense vector learned from word co-occurrence statistics within a large text corpus, allowing semantically similar words to occupy nearby positions in the vector space. A GloVe sentence embedding is computed by averaging the pre-trained word vectors of all words in the sentence as defined in (3) (GloVe Embeddings, p. 23).

	Sentence 1	Sentence 1 Vector (first 10 dims)	Sentence 2	Sentence 2 Vector (first 10 dims)
0	girl styling hair	0.0004, -0.1115, -0.3969, -0.1707, -0.1546, -0.0412, -0.3012, 0.0975, 0.1233, -0.9641	girl brushing hair	-0.1481, -0.0525, -0.3660, -0.0310, -0.0699, -0.2693, -0.3413, 0.0081, 0.3452, -0.8179
1	group men play soccer beach	-0.1784, 0.4674, -0.0862, 0.0117, -0.0760, 0.1066, -0.0111, 0.4522, 0.1880, -0.6972	group boy playing soccer beach	-0.1623, 0.3541, -0.1867, -0.0231, -0.1054, 0.0149, -0.0382, 0.2206, 0.0018, -0.5815
2	one woman measuring another woman 's ankle	-0.2964, 0.1212, -0.2020, -0.2351, -0.1207, -0.0994, -0.2035, -0.2443, -0.2231, -1.1805	woman measure another woman 's ankle	-0.2403, 0.0677, -0.2446, -0.1069, -0.1136, -0.1009, -0.1002, -0.2476, -0.2432, -1.1838
3	man cutting cucumber	-0.0026, -0.1995, 0.2746, 0.0236, 0.1823, -0.0022, 0.0283, 0.0109, -0.2188, -0.7487	man slicing cucumber	-0.0457, -0.3220, 0.2283, -0.0294, 0.1712, 0.0015, -0.0321, -0.1208, -0.1123, -0.2844
4	man playing harp	-0.2527, 0.2195, -0.5054, 0.0012, -0.0378, 0.0421, -0.1995, 0.1627, -0.2144, -0.4426	man playing keyboard	-0.4248, 0.1782, -0.1287, -0.3037, -0.1850, 0.0109, -0.2930, -0.2263, 0.0291, -0.7571

Note: Only the first 5 dimensions are shown for readability; each embedding has 300 dimensions.

Figure 13 GloVe Sentence Embeddings of the First 5 Sentence Pairs from the Test Split Shown in Figure 8

The GloVe sentence embeddings in Figure 13 present dense numerical representations that capture the semantic relationships among words within a sentence. Each numerical value corresponds to a feature in the 300-dimensional vector space, where semantically related words are positioned closer together.

The results reveal that sentences describing similar activities, such as “cutting a cucumber” and “slicing a cucumber,” exhibit closely aligned values in several dimensions, signifying their semantic proximity. Meanwhile, sentences describing unrelated actions show more distinct numerical distributions. These findings confirm that GloVe effectively captures semantic meaning by encoding relationships derived from word co-occurrence statistics as discussed by Pennington et al. [26]. This allows the embeddings to represent overall sentence meaning more comprehensively than simple embedding models.

Generation of DeBERTa Embeddings

To capture deeper semantic and contextual meaning beyond lexical and co-occurrence information, this study utilized DeBERTa v3-base (Decoding-Enhanced BERT with Disentangled Attention). Unlike TF-IDF and GloVe, which assign static weights to words, DeBERTa generates dynamic contextual embeddings where each token’s representation is influenced by surrounding words. A sentence embedding was obtained by taking the mean of the final hidden-state vectors across all tokens as defined in (4) (DeBERTa Embeddings, p. 27).

The code snippet in Figure 14 illustrates how contextual sentence embeddings are extracted using the DeBERTa transformer model. Sentence representations are obtained

by averaging the final hidden-state token embeddings to capture deep contextual and semantic information.

```
# Load the pre-trained DeBERTa model and tokenizer, and define a function to generate
sentence embeddings using mean pooling of token representations.
DEBERTA_MODEL = "microsoft/deberta-v3-base"
tokenizer = AutoTokenizer.from_pretrained(DEBERTA_MODEL)
model = AutoModel.from_pretrained(DEBERTA_MODEL).to(device).eval()

@torch.no_grad()
def embed_text(texts, batch_size=32, max_length=384):
    outs = []
    for i in range(0, len(texts), batch_size, desc="DeBERTa"):
        batch = texts[i:i+batch_size]
        inputs = tokenizer(
            batch, return_tensors="pt", padding=True, truncation=True,
            max_length=max_length
        ).to(device)
        outputs = model(**inputs).last_hidden_state
        mask = inputs["attention_mask"].unsqueeze(-1)
        vec = (outputs * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1)
        outs.append(vec.detach().cpu().numpy().astype(np.float32))
    if device.type == "cuda":
        torch.cuda.empty_cache()
    return np.vstack(outs)

# Generate DeBERTa embeddings for the first 10 test sentence pairs and display the
first 10 dimensions of each sentence vector.

table_rows_deb = []

v1_te = embed_text(s1_raw_te)
v2_te = embed_text(s2_raw_te)

for i in range(10):
    s1, s2 = s1_raw_te[i], s2_raw_te[i]
    v1, v2 = v1_te[i], v2_te[i]
    sim = sim_deb_te[i]

    vec1_str = ", ".join([f"{x:.4f}" for x in v1[:10]])
    vec2_str = ", ".join([f"{x:.4f}" for x in v2[:10]])
```

Note: Only the first 10 dimensions are shown for readability; each embedding has 768 dimensions.

Figure 14 Code Snippet for Generating DeBERTa Embeddings

The DeBERTa embeddings shown in Figure 15 demonstrate the model’s ability to encode contextual meaning through its attention-based mechanism. Each value represents a dimension in a 768-length vector generated by averaging contextual token embeddings. For example, in the pair “A man is cutting up a cucumber” and “A man is slicing a cucumber,” the generated embeddings display small yet meaningful changes across dimensions, indicating that DeBERTa recognizes the contextual relationship between the

two actions. These results highlight the model’s capability to capture deeper linguistic structures and nuanced word dependencies within a sentence, making the embeddings rich in semantic and contextual information essential for similarity estimation, a finding supported by He et al. [13] in their study.

	Sentence 1	Sentence 1 Vector (first 10 dims)	Sentence 2	Sentence 2 Vector (first 10 dims)
0	A girl is styling her hair.	0.1776, 0.0729, -0.0906, 0.2049, -0.1703, -0.3401, 0.1908, 0.0016, -0.0304, -0.1389	A girl is brushing her hair.	0.1032, 0.0144, -0.0838, 0.1594, -0.1521, -0.3513, 0.1420, 0.0158, -0.0112, -0.1178
1	A group of men play soccer on the beach.	0.2289, 0.1055, -0.0383, 0.0951, -0.1149, -0.2617, 0.0973, -0.0772, 0.0996, -0.1390	A group of boys are playing soccer on the beach.	0.1844, 0.2098, -0.0755, 0.1641, -0.1233, -0.2999, 0.0894, -0.0078, 0.1107, -0.1379
2	One woman is measuring another woman's ankle.	0.0525, 0.1276, -0.0880, 0.3460, 0.0279, -0.2929, 0.0361, -0.1295, -0.1121, -0.0488	A woman measures another woman's ankle.	0.0453, -0.0362, 0.0399, 0.1196, -0.0414, -0.4676, 0.0607, -0.2188, 0.1847, -0.0644
3	A man is cutting up a cucumber.	0.2607, -0.0088, -0.1558, 0.2220, -0.2353, -0.0904, 0.1425, 0.0942, 0.0528, 0.1615	A man is slicing a cucumber.	0.1985, -0.0115, -0.1541, 0.1850, -0.2468, -0.1583, 0.2006, 0.1674, 0.0225, 0.0087
4	A man is playing a harp.	0.3119, 0.0347, -0.2075, -0.0329, -0.2451, -0.1381, 0.2030, 0.1094, -0.1147, -0.1548	A man is playing a keyboard.	0.3356, 0.0768, -0.1230, 0.0516, -0.2470, -0.1939, 0.2393, 0.1843, -0.0751, -0.2046

Figure 15 DeBERTa Sentence Embeddings of the First 5 Sentence Pairs from the Test Split Shown in Figure 8

B. Compute cosine similarity scores for each embedding method

After generating the individual embeddings for each sentence using TF-IDF, GloVe, and DeBERTa, cosine similarity was applied to determine the degree of similarity between each pair of sentences. The cosine similarity quantifies how closely two vectors align in an n-dimensional space, as represented by **(8) (Cosine Similarity, p. 31)**.

The code snippet in Figure 16 computes cosine similarity scores for each sentence pair using their TF-IDF, GloVe, and DeBERTa embeddings. These similarity values quantify the degree of alignment between sentence vectors in their respective embedding spaces.

```

# Compute cosine similarity between TF-IDF vectors of sentence pairs.
def tfidf_sim(s1_list, s2_list):
    v1, v2 = tfidf_vectorizer.transform(s1_list), tfidf_vectorizer.transform(s2_list)
    return np.array([cosine_similarity(v1[i], v2[i])[0, 0] for i in
range(v1.shape[0])])
    , dtype="float32")
# Compute cosine similarity between GloVe sentence embeddings of sentence pairs.
def glove_sim(s1_list, s2_list):
    return np.array([
        cosine_similarity([glove_embed_sentence(a,
glove_dim)], [glove_embed_sentence(b,
glove_dim)])][0, 0] for a, b
in zip(s1_list, s2_list)], dtype="float32")
# Compute cosine similarity between DeBERTa sentence embeddings of sentence pairs.
def deberta_sim(s1_list, s2_list):
    v1, v2 = embed_text(s1_list), embed_text(s2_list)
    v1 /= np.linalg.norm(v1, axis=1, keepdims=True)
    v2 /= np.linalg.norm(v2, axis=1, keepdims=True)
    return np.sum(v1 * v2, axis=1).astype("float32")

```

Figure 16 Code Snippet for Computing Cosine Similarity using TF-IDF, GloVe, and DeBERTa Embeddings

	Sentence 1	Sentence 2	TF-IDF Similarity	GloVe Similarity	DeBERTa Similarity	Gold Score (Y _{test})
0	A girl is styling her hair.	A girl is brushing her hair.	0.4828	0.8090	0.9923	0.50
1	A group of men play soccer on the beach.	A group of boys are playing soccer on the beach.	0.6733	0.9143	0.9919	0.72
2	One woman is measuring another woman's ankle.	A woman measures another woman's ankle.	0.6895	0.9580	0.9807	1.00
3	A man is cutting up a cucumber.	A man is slicing a cucumber.	0.6943	0.8807	0.9893	0.84
4	A man is playing a harp.	A man is playing a keyboard.	0.3281	0.8060	0.9944	0.30

Figure 17 Cosine Similarity Scores of the First 5 Sentence Pairs from the Test Split Shown in Figure 8

For the TF-IDF model, the similarity scores reflect how much lexical overlap exists between two sentences. Sentence pairs that share multiple identical or closely related words, such as “man cutting cucumber” and “man slicing cucumber,” produced higher scores (around 0.70), showing that TF-IDF effectively captures the presence of

common keywords. Meanwhile, pairs with fewer shared terms, such as “man playing harp” and “man playing keyboard,” yielded lower scores (around 0.33), indicating limited lexical correspondence. These results confirm that TF-IDF focuses on literal word matching and successfully quantifies similarity when overlapping vocabulary is present, a behavior also noted by Lan in their study [17] .

For the GloVe model, the results show both strengths and limitations. On the positive side, GloVe successfully recognizes semantic relationships even when the exact words differ. For example, the pair “group men play soccer beach” and “group boy playing soccer beach” reached a high similarity score of 0.9143, indicating that the model identifies the conceptual link between “men” and “boy,” which aligns with the findings of Pennington et al. [26]. However, GloVe also produced relatively high scores for sentences that are not entirely the same in meaning. For instance, “man playing harp” and “man playing keyboard” received a similarity value of 0.8060 despite describing distinct activities. This suggests that GloVe tends to overestimate similarity for semantically related but contextually different sentences, a limitation also noted by Hindocha et al. [34], as it measures general word co-occurrence rather than precise meaning.

For the DeBERTa model, the cosine similarity scores were consistently high, often nearing 1.0, even when sentence meanings were not fully identical. For example, “A girl is styling her hair” and “A girl is brushing her hair” obtained a similarity value of 0.9923. Although both sentences describe a similar action, they involve different objects, yet the model treated them as almost equivalent. This pattern suggests that DeBERTa’s contextual representations may overestimate semantic similarity, which is consistent with Zou et al.’s [15] finding that the pre-trained DeBERTa model exhibits strong anisotropy,

where the distance between any two samples is close. While the model captures overall sentence structure and intent well, its high similarity outputs suggest a need to fine-tune the model to reduce overestimation of similarity.

Overall, the cosine similarity computations show that while each embedding effectively transforms text into numerical form, both GloVe and DeBERTa occasionally assign excessively high similarity values even for non-identical sentences. This overestimation highlights the importance of further refinement through regression modeling and ensemble weighting to better align machine-computed similarity with human judgment.

C. Form a feature vector from the three similarity scores

The final feature vector for each sentence pair is constructed by combining the cosine similarity scores derived from the three embedding models as defined in (12) (Feature Vector, p. 42).

```
# Combine TF-IDF, GloVe, and DeBERTa similarity scores into feature vectors.
X_train = np.stack([sim_tfidf_tr, sim_glove_tr, sim_deb_tr],
axis=1).astype("float32")
X_dev   = np.stack([sim_tfidf_de, sim_glove_de, sim_deb_de],
axis=1).astype("float32")
X_test  = np.stack([sim_tfidf_te, sim_glove_te, sim_deb_te],
axis=1).astype("float32")

Y_train, Y_dev, Y_test = map(lambda y: y.astype("float32"), [y_train, y_dev, y_test])
```

Figure 18 Code Snippet for Constructing the Combined Feature Vector from TF-IDF, GloVe, and DeBERTa Similarity Scores

The code snippet shown in Figure 18 constructs the final feature vector by combining the cosine similarity scores from TF-IDF, GloVe, and DeBERTa. Each sentence pair is represented as a three-dimensional vector that serves as input to the regression model.

The feature vector shown in Figure 19 combines the cosine similarity values from the three embedding methods TF-IDF, GloVe, and DeBERTa into a single representation. Each sentence pair is therefore represented by three numerical features along with the gold score as reference.

	x1 = TF-IDF	x2 = GloVe	x3 = DeBERTa
0	0.4828	0.8090	0.9923
1	0.6733	0.9143	0.9919
2	0.6895	0.9580	0.9807
3	0.6943	0.8807	0.9893
4	0.3281	0.8060	0.9944

Figure 19 Feature Vector of the First 5 Sentence Pairs from the Test Split Shown in Figure 8

The results illustrate that each feature contributes distinct information to the overall similarity measurement. In the pair “one woman measuring another woman’s ankle” and “a woman measures another woman’s ankle,” all three features show relatively high scores (TF-IDF = 0.6895, GloVe = 0.9580, DeBERTa = 0.9807, Gold = 1.00). While GloVe and DeBERTa are close to the gold score, TF-IDF yields a noticeably lower value. This indicates that TF-IDF, being sensitive to lexical differences, underestimates similarity when sentences use slightly different wording, a limitation observed by Qaiser and Ali in their study [4].

However, this conservative behavior of TF-IDF can also be advantageous in detecting dissimilarity. For instance, in “man playing harp” and “man playing keyboard,” the TF-IDF score (0.3281) closely aligns with the gold score (0.30), correctly reflecting

that the two sentences describe different actions. In contrast, GloVe and DeBERTa give higher scores (0.8060 and 0.9944), showing that semantic and contextual embeddings may overestimate similarity when sentences are related but not equivalent in meaning, a limitation noted across the studies of Hindocha et al. [34] and Zou et al. [15].

These observations show that each embedding captures a unique dimension of similarity: term importance from TF-IDF, global relationship from GloVe, and contextual information from DeBERTa. By combining these features into one vector, the ensemble model can learn how to weigh and balance them appropriately. This combination enables the system to reduce individual model biases and produce similarity predictions that better align with human judgments.

D. Train a Ridge Regression model to learn the optimal weighted combination of the similarity scores for predicting semantic similarity

To fulfill the fourth objective, the Ridge Regression ensemble model was trained and evaluated using the normalized STS-B English dataset. The model integrates the cosine similarity scores computed from TF-IDF, GloVe, and DeBERTa embeddings to form a unified feature representation for each sentence pair.

Following **(6) (Ridge Prediction Function, p. 30)**, the three similarity scores from TF-IDF, GloVe, and DeBERTa embeddings are combined into a weighted feature vector using their learned coefficients ($w_{\text{tfidf}} = 0.7193$, $w_{\text{glove}} = 0.1950$, $w_{\text{deberta}} = 0.2812$) and a bias term of -0.2274 . Parameter optimization followed the loss function presented in **(7) (Ridge Loss Function, p. 30)**. Hyperparameter tuning was conducted through 8-fold cross-validation to determine the optimal Ridge penalty

coefficient (λ). The best performance was achieved at $\lambda = 0.0001$ with a patience value of 8, yielding a mean validation Pearson correlation of 0.7386 ± 0.0171 , indicating that moderate regularization provided the best trade-off between bias and variance. This selection is consistent with prior studies suggesting that very small L2 regularization strengths impose only mild constraints on model coefficients [41]. The implementation used for Ridge regression training and hyperparameter tuning is provided in **Appendix A. 2 (p. 87)**, **A. 3 (p. 87)**, and **A. 3 (p. 88)**.

```
# Define a Ridge Regression model with L2 regularization for similarity prediction.
def make_ridge_model(input_dim=3, lambda_l2=0.01, use_bias=True, lr=1e-2):
    inp = layers.Input(shape=(input_dim,), name="features")
    out = layers.Dense(
        1,
        activation="linear",
        use_bias=use_bias,
        kernel_regularizer=regularizers.l2(lambda_l2),
        name="ridge_head",
    )(inp)
    model = models.Model(inp, out, name="ridge_similarity")
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=lr),
        loss="mse",
        metrics=["mae"]
    )
    return model
```

Figure 20 Code Snippet for Training a Ridge Regression Model

The code snippet shown in Figure 20 presents the training procedure for the Ridge Regression ensemble model. It learns the optimal weighted combination of the three similarity features while applying L2 regularization to prevent overfitting and improve generalization.

The validation loss stabilized around $\text{val_loss} \approx 0.0360$ and $\text{val_mae} \approx 0.1499$ after 10 epochs, confirming good generalization performance. The loss and MAE values consistently decreased over 10 epochs, indicating effective convergence and

generalization of the TGD (Ensemble) model. The minimal gap between training and validation metrics confirms stable learning behavior without signs of overfitting.

Epoch	Training Loss (MSE)	Training MAE	Validation Loss (MSE)	Validation MAE
1	0.178700	0.338422	0.083395	0.238968
2	0.077919	0.233378	0.051623	0.188300
3	0.055100	0.195740	0.040598	0.165972
4	0.046705	0.177579	0.038530	0.159057
5	0.044045	0.170119	0.038288	0.156431
6	0.042905	0.166577	0.037958	0.154690
7	0.042100	0.164394	0.037451	0.153243
8	0.041418	0.162764	0.036921	0.151879
9	0.040839	0.161426	0.036446	0.150754
10	0.040362	0.160302	0.036048	0.149883

Figure 21 Training and Validation Loss and MAE per Epoch TGD (Ensemble)

Figure 22 illustrates the consistent decrease in training and validation MSE across epochs. Both curves rapidly converged and flattened by the tenth epoch, demonstrating stable convergence and absence of overfitting.

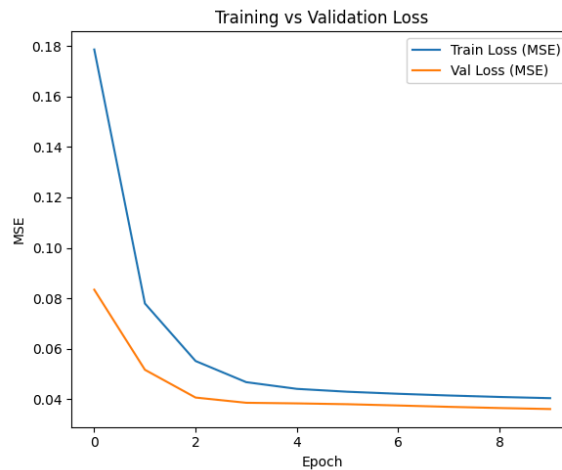


Figure 22 Training and Validation Loss (MSE) Curves of the TGD (Ensemble)

As shown in Figure 23, both training and validation MAE values steadily declined and reached similar error levels near the end of training.

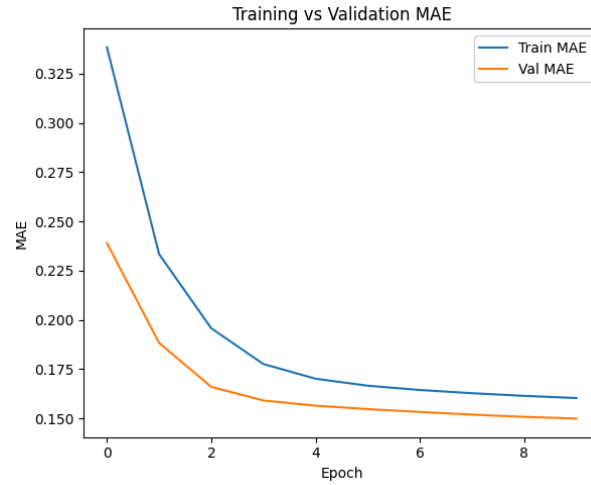


Figure 23 Training and Validation Mean Absolute Error (MAE) Curves of the TGD (Ensemble)

E. Evaluate the ensemble model against standalone TF-IDF, GloVe, and DeBERTa models using MSE, MAE, Pearson Correlation, Accuracy, and F1-score

To further evaluate the model's predictive performance, evaluation metrics were employed, including MSE, MAE, Accuracy, and F1-score. These measures collectively evaluate how well the model distinguishes between similar and dissimilar sentence pairs using a similarity-score threshold of 0.5. The implementation code used to generate the evaluation metrics is in **Appendix A.5 (p. 92)**.

Table VII Evaluation performance of standalone and ensemble models

Model	PCC	MSE	MAE	Accuracy	F1-Score
TF-IDF	0.6622	0.0602	0.1798	0.7302	0.7958

GloVe	0.5390	0.1367	0.3025	0.7259	0.7845
DeBERTa	0.1984	0.2766	0.4376	0.5823	0.6962
TGD (Ensemble)	0.6697	0.0517	0.1773	0.7578	0.8080

In Table VII, the TF-IDF model achieved a PCC of 0.6622, with an $MSE = 0.0602$ and $MAE = 0.1798$, indicating a moderately strong linear relationship between predicted and gold scores according to the interpretation guidelines of Schober et al. [54]. These results suggest that TF-IDF provides a reasonable level of alignment with human similarity judgments, although the remaining error reflects the limitations of relying solely on lexical overlap. Previous studies report similar behavior. For instance, Yulita et al. [19] noted that TF-IDF-based cosine similarity approaches are sensitive to lexical variation even when texts are semantically equivalent. Similarly, King et al. [23] demonstrated that lexical overlap can provide a strong yet imperfect signal for semantic similarity in STS tasks. Supporting this observation, Isbister and Sahlgren [24] found that simple bag-of-words and TF-IDF representations often perform competitively on STS-B-derived datasets, sometimes rivaling more complex unsupervised semantic models. Additional evidence from Silva Junior et al. [25] and Arroyo-Fernández et al. [8] further confirms that TF-IDF remains a reliable unsupervised baseline across various STS domains. Consistent with these findings, the TF-IDF model in this study demonstrated relatively strong performance, although it remained slightly below the ensemble model. Its high F1-score indicates that lexical-based representations remain effective for capturing surface-level similarity patterns, particularly when sentences share common terms, as also noted by Lan [17]. Overall, these findings highlight that while TF-IDF can achieve competitive performance and relatively low error among baseline models, its

reliance on surface-level lexical similarity limits its ability to capture deeper contextual semantics.

In contrast, the GloVe model achieved a test PCC of 0.5390, which falls within the moderate correlation range defined by Schober et al. [54]. Its test $MSE = 0.1367$ and $MAE = 0.3025$ indicate that while global co-occurrence embeddings moderately improved the model’s alignment with human-annotated scores, performance remained constrained by GloVe’s static embedding structure. These findings are consistent with those of Malik et al. [30], who evaluated GloVe embeddings for semantic modeling and observed similar performance behavior in semantic similarity tasks. Similarly, studies by Zhelezniak [31] and Kim et al. [32] reported that GloVe-based representations typically achieve moderate correlation levels in semantic similarity evaluations, supporting the trend observed in this study. While the GloVe model captured broader semantic associations through global co-occurrence patterns and produced relatively balanced results, it still fell short of the ensemble model’s stability. As highlighted by Hindocha et al. [34], GloVe relies on fixed word vectors that do not adapt to contextual variations, limiting its ability to interpret sentences that express similar meanings using different wording. Consequently, although GloVe effectively captures overall semantic relationships, its lack of contextual sensitivity and inability to handle polysemy contribute to moderate correlations and residual prediction errors, reflecting the performance trends observed in this study.

The DeBERTa model recorded the weakest performance among all baselines, with a test PCC of 0.1984, indicating a weak relationship according to the classification of Schober et al. [54]. Its $MSE = 0.2766$ and further demonstrate substantial prediction

deviations from human-annotated scores. Similarly, the model achieved the lowest values in accuracy (0.5823) and F1-score (0.6962), reinforcing its relatively poor alignment with the STS-B similarity judgments. Although DeBERTa is designed to capture deep contextual meaning, it tends to overemphasize semantic context while overlooking surface-level lexical cues that are important in the STS-B scoring framework. This imbalance can reduce its ability to accurately reflect graded semantic similarity. These findings are consistent with the results reported by Kowsher et al. [39], who also observed weak STS-B performance in DeBERTa-based models. Furthermore, Li et al. [14] and Zou et al. [15] attribute this limitation partly to anisotropic embedding spaces, where semantic representations are unevenly distributed, leading to poor alignment with similarity evaluation metrics. As a result, contextual models such as DeBERTa-v3 alone may struggle to produce balanced similarity estimates, particularly in cases where subtle lexical distinctions play an important role.

The TGD (Ensemble) model achieved the best overall performance across all evaluation measures, demonstrating the strongest alignment with human similarity scores. It recorded the highest correlation scores and lowest error values, with a test PCC of 0.6697, that falls within the moderate yet competitive range described by Schober et al. [54] for interpreting correlation coefficients. The low $MSE = 0.0517$ and $MAE = 0.1773$ indicate that the predicted similarity scores closely matched human-annotated scores. In addition, the model obtained the highest accuracy (0.7578) and an F1-score (0.8080), demonstrating its effectiveness in distinguishing semantically similar and dissimilar sentence pairs. The substantial reduction in prediction errors suggests that combining TF-IDF, GloVe, and DeBERTa similarity scores through Ridge

Regression improved the model’s stability and predictive accuracy. This outcome is consistent with the findings of Huang et al. [40], who reported that combining multiple pre-trained models through ensemble regression techniques can significantly improve semantic similarity performance. Similar evidence was reported in the UNBNLP study by King et al. [23], where Ridge Regression was used to integrate multiple baseline and embedding-based STS approaches. These results further support the effectiveness of combining multiple similarity measures through Ridge Regression to enhance performance in semantic textual similarity tasks.

The similarity scores were converted into binary values using predefined thresholds. The gold similarity scores were binarized using a threshold of 0.5, where sentence pairs with scores equal to or greater than 0.5 were assigned a label of 1, while scores below 0.5 were assigned a label of 0. Similarly, the cosine similarity scores generated by each model were converted into binary labels using their respective optimal thresholds. The threshold applied for TF-IDF was 0.25, while GloVe used 0.748, DeBERTa used 0.929, and the TGD (Ensemble) model used 0.4175. These thresholds were used to convert similarity scores into binary values to enable direct comparison with the binarized reference labels. The implementation used to perform this binarization is provided in **Appendix A.5 (p. 92)**, while the resulting binary labels are presented in the corresponding tables.

	sentence1	sentence2	gold score	gold label	tf-idf score	tf-idf label
0	A girl is styling her hair.	A girl is brushing her hair.	0.50	1	0.48	1
1	A group of men play soccer on the beach.	A group of boys are playing soccer on the beach.	0.72	1	0.67	1
2	One woman is measuring another woman's ankle.	A woman measures another woman's ankle.	1.00	1	0.69	1
3	A man is cutting up a cucumber.	A man is slicing a cucumber.	0.84	1	0.69	1
4	A man is playing a harp.	A man is playing a keyboard.	0.30	0	0.33	1

Figure 24 Binary Label Representation of TF-IDF Similarity Scores

	sentence1	sentence2	gold score	gold label	glove score	glove label
0	A girl is styling her hair.	A girl is brushing her hair.	0.50	1	0.81	1
1	A group of men play soccer on the beach.	A group of boys are playing soccer on the beach.	0.72	1	0.91	1
2	One woman is measuring another woman's ankle.	A woman measures another woman's ankle.	1.00	1	0.96	1
3	A man is cutting up a cucumber.	A man is slicing a cucumber.	0.84	1	0.88	1
4	A man is playing a harp.	A man is playing a keyboard.	0.30	0	0.81	1

Figure 25 Binary Label Representation of GloVe Similarity Scores

	sentence1	sentence2	gold score	gold label	deberta score	deberta label
0	A girl is styling her hair.	A girl is brushing her hair.	0.50	1	0.99	1
1	A group of men play soccer on the beach.	A group of boys are playing soccer on the beach.	0.72	1	0.99	1
2	One woman is measuring another woman's ankle.	A woman measures another woman's ankle.	1.00	1	0.98	1
3	A man is cutting up a cucumber.	A man is slicing a cucumber.	0.84	1	0.99	1
4	A man is playing a harp.	A man is playing a keyboard.	0.30	0	0.99	1

Figure 26 Binary Label Representation of DeBERTa Similarity Scores

	sentence1	sentence2	gold score	gold label	tgd score	tgd label
0	A girl is styling her hair.	A girl is brushing her hair.	0.50	1	0.56	1
1	A group of men play soccer on the beach.	A group of boys are playing soccer on the beach.	0.72	1	0.71	1
2	One woman is measuring another woman's ankle.	A woman measures another woman's ankle.	1.00	1	0.73	1
3	A man is cutting up a cucumber.	A man is slicing a cucumber.	0.84	1	0.72	1
4	A man is playing a harp.	A man is playing a keyboard.	0.30	0	0.45	1

Figure 27 Binary Label Representation of TGD (Ensemble) Similarity Scores

A confusion matrix table is generated to present the relationship between the predicted labels and reference labels. It provides a detailed breakdown of correct and incorrect predictions by showing the number of true positives, true negatives, false positives, and false negatives. This allows for a clearer analysis of the model's prediction performance and helps determine how well the model distinguishes between semantically similar and dissimilar sentence pairs.

Table VIII Confusion matrix

Model	TN	FP	FN	TP
TF-IDF	282	325	47	725
GloVe	313	294	85	687
DeBERTa	143	464	112	660
TGD (Ensemble)	342	265	69	703

The TGD (Ensemble) model demonstrated the most balanced performance, correctly identifying 703 true positives and 342 true negatives, outperforming all baseline models in both categories. Its false-positive count (265) was lower than that of the other models, indicating a moderate improvement in distinguishing between truly similar and dissimilar sentence pairs. This finding is consistent with the general efficacy of ensemble methods reported by Huang et al. [40], who demonstrated that combining multiple pre-trained models with ensemble methods can significantly enhance overall performance. In contrast, the TF-IDF baseline demonstrated competitive performance but was still slightly lower than the ensemble model. Moreover, Isbister and Sahlgren [24] showed that simple bag-of-words and TF-IDF representations perform remarkably well on STS-B-derived data, which is consistent with the strong recall observed for the TF-IDF baseline in this study. GloVe showed a more balanced outcome with fewer false positives (294) but higher false negatives (84), suggesting that although it effectively captures global co-occurrence patterns, it may still miss subtle or paraphrased similarities, an observation consistent with Hindocha et al. [34] who describe GloVe’s reliance on fixed word representations. Meanwhile, DeBERTa exhibited the weakest behavior, with both high false positives (464) and false negatives (112). This can be attributed to known limitations in DeBERTa’s anisotropic embedding space, where

sentence representations are compressed and unevenly distributed, reducing class separability. As reported by Li et al. [14] and Zou et al. [15], such characteristics can hinder fine-grained similarity discrimination even after supervised fine-tuning, helping explain the misclassification observed on the STS-B dataset. These results collectively emphasize that combining contextual and lexical features, as in the TGD (Ensemble), produces more accurate and stable similarity predictions.

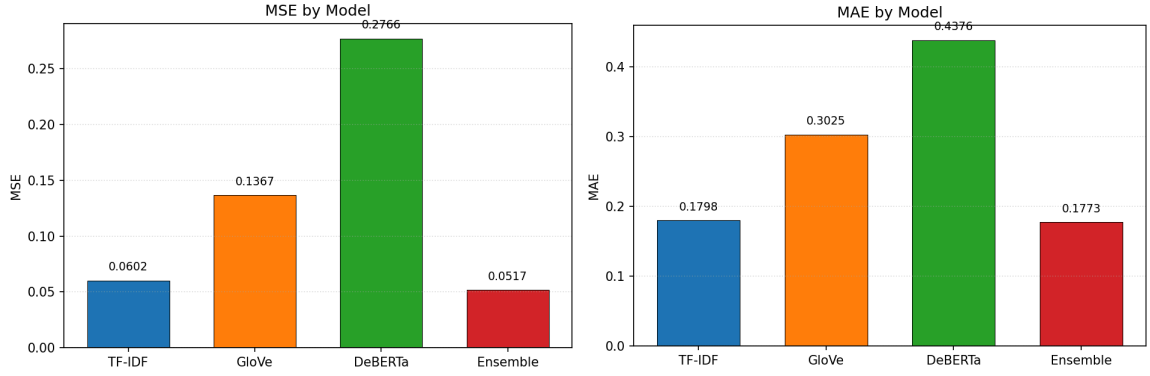


Figure 28 Test Split Mean Squared Error (MSE) and Mean Absolute Error (MAE) of the TGD (Ensemble) Model

The bar graphs illustrate the Mean Squared Error (MSE) and Mean Absolute Error (MAE) values for each model on the STS-B test split. Among the baseline models,

DeBERTa exhibited the highest error values of and $MAE = 0.4376$, indicating that its contextual embeddings alone could not produce stable similarity predictions. GloVe followed with $MSE = 0.1367$ and $MAE = 0.3025$, while TF-IDF showed slightly lower error rates of $MSE = 0.0602$ and $MAE = 0.1798$. In contrast, the Ridge Ensemble model achieved the lowest errors overall, $MSE = 0.0517$ and $MAE = 0.1773$, confirming its superior predictive consistency. The reduced error magnitudes demonstrate that combining lexical, semantic, and contextual features enabled the ensemble to generate similarity scores that were more closely aligned with

human-annotated scores. $MSE = 0.4376$, indicating that its contextual embeddings alone could not produce stable similarity predictions. GloVe followed with $MSE = 0.1367$ and $MAE = 0.3025$, while TF-IDF showed slightly lower error rates of $MSE = 0.0602$ and $MAE = 0.1798$. In contrast, the Ridge Ensemble model achieved the lowest errors overall, $MSE = 0.0517$ and $MAE = 0.1773$, confirming its superior predictive consistency. The reduced error magnitudes demonstrate that combining lexical, semantic, and contextual features enabled the ensemble to generate similarity scores that were more closely aligned with human-annotated scores.

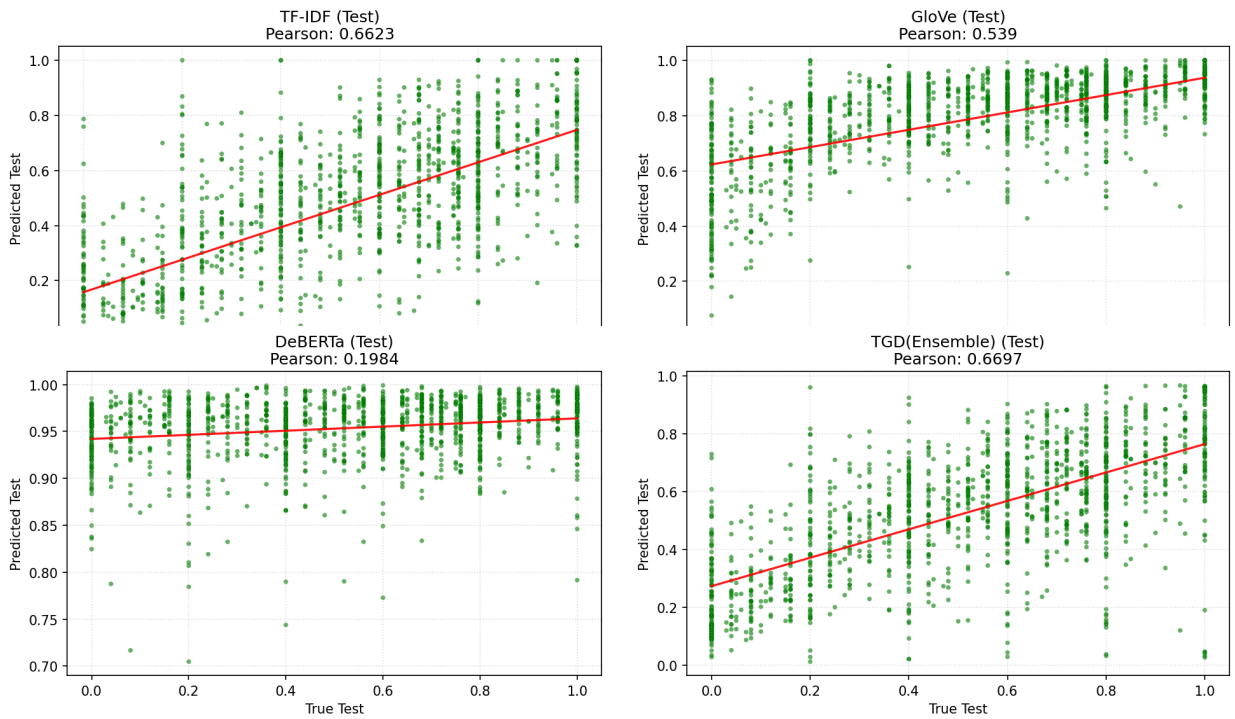


Figure 29 Correlation Scatter Plots of Predicted vs. Actual Similarity Scores

The scatter plots illustrate the relationship between the predicted and true similarity scores for all models. Each subplot includes a red regression line indicating how closely the model's predictions align with the human-annotated scores. The TGD

(Ensemble) model demonstrates the strongest PCC of 0.6697, with points densely clustered around the regression line, showing high consistency and reliable prediction of semantic similarity. TF-IDF and GloVe models show moderate linear relationships (0.6623 and 0.5390, respectively), suggesting that while lexical and semantic features capture general similarity patterns, they are less precise in representing subtle relationships, which reflects findings reported by studies by King [23], Zhelezniak [31], and Kim et al. [32]. In contrast, the DeBERTa model exhibits the weakest correlation, with predictions scattered widely from the trend line, indicating that contextual embeddings alone were less effective in aligning with the STS-B test scoring. This limitation is consistent with the findings of Li et al. [14] and Zou et al. [15], who reported that anisotropic embedding spaces in DeBERTa compress semantic distances. These results confirm that integrating multiple embeddings using Ridge Ensemble yields more accurate and human-consistent similarity predictions.

CHAPTER V

SUMMARY, CONCLUSION, AND RECOMMENDATION

This chapter presents the summary, conclusions, and recommendations of the study. It synthesizes the key findings derived from evaluation of the proposed weighted ensemble model, highlighting its performance and contributions to the field of semantic textual similarity. Furthermore, this chapter provides conclusions based on the results and offers recommendations for future research and potential improvements to the model.

A. Summary

This study developed a weighted ensemble model that integrates Term Frequency-Inverse Document Frequency (TF-IDF), Global Vector for Word Representation (GloVe,) and DeBERTa contextual embeddings through Ridge Regression to improve semantic textual similarity prediction. The general objective was to improve semantic similarity prediction by combining cosine similarity scores from TF-IDF, GloVe, and DeBERTa embeddings using a regression-based ensemble model. Specifically, the study aimed to (1) generate TF-IDF, GloVe, and DeBERTa embeddings; (2) compute cosine similarity scores for each embedding; (3) form a feature vector from the three similarity scores; (4) train a Ridge Regression ensemble model to determine the optimal weighted combination of features; and (5) evaluate the ensemble model against standalone TF-IDF, GloVe, and DeBERTa models using MSE, MAE, Pearson Correlation, Accuracy, and F1-score.

The normalized English Semantic Textual Similarity Benchmark (STS-B) dataset from the Hugging Face repository, consisting of 8,628 sentence pairs, served as the experimental corpus. The dataset was divided into training (5,749 sentence pairs),

development (1,500 sentence pairs), and test splits (1,379 sentence pairs), allowing systematic model training, validation, and evaluation. Preprocessing included lowercasing, tokenization, stop-word removal, and lemmatization for TF-IDF and GloVe inputs, while DeBERTa utilized raw sentence pairs to retain syntactic and contextual structure. Each sentence was represented through its respective embedding method, and cosine similarity was computed between sentence pairs. The resulting similarity scores were concatenated into a three-dimensional feature vector that served as input to the Ridge Regression model. Hyperparameter optimization through 8-fold cross-validation identified the optimal regularization coefficient $\lambda = 0.0001$, which achieved a balanced trade-off between bias and variance.

Experimental results showed that the TGD (Ensemble) outperformed all individual baseline models. It achieved a test Pearson correlation of 0.6697, $MSE = 0.0517$, and $MAE = 0.1773$, demonstrating a strong linear relationship between predicted and gold similarity scores. Using a threshold of 0.5, the ensemble obtained Accuracy of 75.78%, and F1-score of 80.80%, confirming the model's balanced capability in identifying semantically similar and dissimilar sentence pairs.

B. Conclusions

The following conclusions were derived from the results of the study:

1. TF-IDF, GloVe, and DeBERTa embeddings were successfully generated, each capturing distinct aspects of sentence meaning. TF-IDF captures lexical importance, GloVe captures global semantic associations, and DeBERTa captures contextual relationships while exhibiting a tendency to overestimate similarity,

confirming that each method captures distinct linguistic features from the normalized STS-B dataset.

2. Cosine similarity scores computed for each embedding method effectively quantified sentence-level semantic relatedness. Each embedding produced meaningful similarity patterns with varying sensitivity to lexical, semantic, or contextual differences.
3. A combined feature vector integrating the three cosine similarity scores was successfully constructed. This unified representation merged lexical, semantic, and contextual information into a single input suitable for regression-based prediction.
4. The Ridge Regression ensemble model effectively learned the optimal weighted combination of similarity features. Cross-validation identified $\lambda = 0.0001$ as the best regularization value, and the model showed stable convergence with low generalization error.
5. The TGD (Ensemble) model outperformed all standalone models in the overall evaluation. Its superior correlation, lower error rates, and higher accuracy and F1-score validate that integrating TF-IDF, GloVe, and DeBERTa similarity scores within a weighted ensemble framework improves semantic similarity prediction more effectively than any single embedding model.

C. Recommendations

- Future studies may apply the proposed TGD (Ensemble) to multilingual or specialized versions of the STS-B dataset to examine its generalization capability

across different languages and contextual domains, since this study was limited to the English normalized variant.

- It is recommended to fine-tune the DeBERTa-V3-Base model on the STS-B corpus or similar text collections to enhance contextual sensitivity and reduce the model's tendency to overestimate similarity in near-paraphrased but semantically distinct sentences.
- Future researchers may investigate non-linear or hybrid ensemble methods such as Gradient Boosting to model complex relationships among TF-IDF, GloVe, and DeBERTa similarity features, potentially yielding higher predictive accuracy.
- To strengthen interpretability and ensure alignment with human judgment, it is recommended to perform manual or crowd-sourced similarity assessments comparing human and model-generated scores on a subset of sentence pairs.
- Subsequent studies should employ high-performance hardware or optimized frameworks to enable deeper experimentation with larger datasets and model variants, overcoming the GPU and resource constraints identified during this study.

Portions of the summary, conclusion, and recommendation were improved with the assistance of OpenAI's ChatGPT (GPT-5.1). All AI-generated content was reviewed, validated, and edited by the researchers to ensure accuracy and alignment with the study's findings [59].

REFERENCES

- [1] L. Zhang, S. R. Wilson, and R. Mihalcea, “Multi-Label Transfer Learning for Multi-Relational Semantic Similarity,” in *Proceedings of the Eighth Joint Conference on Lexical and Computational Semantics (*SEM 2019)*, 2019, pp. 44–50. doi: 10.18653/v1/S19-1005.
- [2] A. Vasileiou and O. Eberle, “Explaining Text Similarity in Transformer Models,” May 10, 2024, *arXiv*: arXiv:2405.06604. doi: 10.48550/arXiv.2405.06604.
- [3] L. Gao, Z. Liu, and Q. Zhang, “A Comprehensive Framework for Semantic Similarity Analysis of Human and AI-Generated Text Using Transformer Architectures and Ensemble Techniques,” Jan. 31, 2025, *arXiv*: arXiv:2501.14288. doi: 10.48550/arXiv.2501.14288.
- [4] S. Qaiser and R. Ali, “Text Mining: Use of TF-IDF to Examine the Relevance of Words to Documents,” *Int. J. Comput. Appl.*, vol. 181, no. 1, pp. 25–29, Jul. 2018, doi: 10.5120/ijca2018917395.
- [5] B. Bakiyev, “Method for Determining the Similarity of Text Documents for the Kazakh language, Taking Into Account Synonyms: Extension to TF-IDF,” in *2022 International Conference on Smart Information Systems and Technologies (SIST)*, Nur-Sultan, Kazakhstan: IEEE, Apr. 2022, pp. 1–6. doi: 10.1109/SIST54437.2022.9945747.
- [6] M. Das, “A Comparative Study on TF-IDF feature Weighting Method and its Analysis using Unstructured Dataset”.
- [7] Zhixiang, Xu, M. Chen, K. Q. Weinberger, and F. Sha, “An alternative text representation to TF-IDF and Bag-of-Words,” Jan. 28, 2013, *arXiv*: arXiv:1301.6770. doi: 10.48550/arXiv.1301.6770.
- [8] I. Arroyo-Fernández, C.-F. Méndez-Cruz, G. Sierra, J.-M. Torres-Moreno, and G. Sidorov, “Unsupervised Sentence Representations as Word Information Series: Revisiting TF--IDF,” Oct. 20, 2017, *arXiv*: arXiv:1710.06524. doi: 10.48550/arXiv.1710.06524.
- [9] Z. Jiang, B. Gao, Y. He, Y. Han, P. Doyle, and Q. Zhu, “Text Classification Using Novel Term Weighting Scheme-Based Improved TF-IDF for Internet Media Reports,” *Math. Probl. Eng.*, vol. 2021, pp. 1–30, Mar. 2021, doi: 10.1155/2021/6619088.
- [10] W. J. Wilbur and W. Kim, “The ineffectiveness of within-document term frequency in text classification,” *Inf. Retr.*, vol. 12, no. 5, pp. 509–525, Oct. 2009, doi: 10.1007/s10791-008-9069-5.
- [11] S. Li and B. Gong, “Word embedding and text classification based on deep learning methods,” *MATEC Web Conf.*, vol. 336, p. 06022, 2021, doi: 10.1051/mateconf/202133606022.
- [12] C. Wang, P. Nulty, and D. Lillis, “A Comparative Study on Word Embeddings in

- Deep Learning for Text Classification,” in *Proceedings of the 4th International Conference on Natural Language Processing and Information Retrieval*, Seoul Republic of Korea: ACM, Dec. 2020, pp. 37–46. doi: 10.1145/3443279.3443304.
- [13] P. He, X. Liu, J. Gao, and W. Chen, “DeBERTa: Decoding-enhanced BERT with Disentangled Attention,” Oct. 06, 2021, *arXiv*: arXiv:2006.03654. doi: 10.48550/arXiv.2006.03654.
- [14] B. Li, H. Zhou, J. He, M. Wang, Y. Yang, and L. Li, “On the Sentence Embeddings from Pre-trained Language Models,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Online: Association for Computational Linguistics, 2020, pp. 9119–9130. doi: 10.18653/v1/2020.emnlp-main.733.
- [15] A. Zou, W. Hao, D. Jin, G. Chen, and F. Sun, “MoCoUTRL: a momentum contrastive framework for unsupervised text representation learning,” *Connect. Sci.*, vol. 35, no. 1, p. 2221406, Dec. 2023, doi: 10.1080/09540091.2023.2221406.
- [16] A. Widiyanto, E. Pebriyanto, F. Fitriyanti, and M. Marna, “Document Similarity Using Term Frequency-Inverse Document Frequency Representation and Cosine Similarity,” *J. Dinda Data Sci. Inf. Technol. Data Anal.*, vol. 4, no. 2, pp. 149–153, Aug. 2024, doi: 10.20895/dinda.v4i2.1589.
- [17] F. Lan, “Research on Text Similarity Measurement Hybrid Algorithm with Term Semantic Information and TF-IDF Method,” *Adv. Multimed.*, vol. 2022, pp. 1–11, Apr. 2022, doi: 10.1155/2022/7923262.
- [18] A. Mutsaddi and A. Choudhary, “Enhancing Plagiarism Detection in Marathi with a Weighted Ensemble of TF-IDF and BERT Embeddings for Low-Resource Language Processing”.
- [19] W. Yulita, M. C. Untoro, M. Praseptiawan, I. F. Ashari, A. Afriansyah, and A. N. Bin Che Pee, “Automatic Scoring Using Term Frequency Inverse Document Frequency Document Frequency and Cosine Similarity,” *Sci. J. Inform.*, vol. 10, no. 2, pp. 93–104, Apr. 2023, doi: 10.15294/sji.v10i2.42209.
- [20] L. Xiang, “Application of an Improved TF-IDF Method in Literary Text Classification,” *Adv. Multimed.*, vol. 2022, pp. 1–10, May 2022, doi: 10.1155/2022/9285324.
- [21] P. M. Hasugian, J. Manurung, and U. Ram, “IMPLEMENTATION OF TF-IDF AND COSINE SIMILARITY ALGORITHMS FOR CLASSIFICATION OF DOCUMENTS BASED ON ABSTRACT SCIENTIFIC JOURNALS,” vol. 9, no. 2, 2021.
- [22] H. Fan and Y. Qin, “Research on Text Classification Based on Improved TF-IDF Algorithm,” in *Proceedings of the 2018 International Conference on Network, Communication, Computer Engineering (NCCE 2018)*, Chongqing, China: Atlantis Press, 2018. doi: 10.2991/ncce-18.2018.79.
- [23] M. King, W. Gharbieh, S. Park, and P. Cook, “UNBNLP at SemEval-2016 Task 1:

- Semantic Textual Similarity: A Unified Framework for Semantic Processing and Evaluation,” in *Proceedings of the 10th International Workshop on Semantic Evaluation (SemEval-2016)*, San Diego, California: Association for Computational Linguistics, 2016, pp. 732–735. doi: 10.18653/v1/S16-1113.
- [24] T. Isbister and M. Sahlgren, “Why Not Simply Translate? A First Swedish Evaluation Benchmark for Semantic Similarity”.
- [25] D. D. Silva Junior, D. D. Oliveira, and A. Paes, “Evaluating text representations for unsupervised legal semantic textual similarity in Brazilian Portuguese,” *Discov. Data*, vol. 3, no. 1, p. 23, Jun. 2025, doi: 10.1007/s44248-025-00052-4.
- [26] J. Pennington, R. Socher, and C. Manning, “Glove: Global Vectors for Word Representation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Doha, Qatar: Association for Computational Linguistics, 2014, pp. 1532–1543. doi: 10.3115/v1/D14-1162.
- [27] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient Estimation of Word Representations in Vector Space,” Sep. 07, 2013, *arXiv*: arXiv:1301.3781. doi: 10.48550/arXiv.1301.3781.
- [28] A. Joulin, E. Grave, P. Bojanowski, M. Douze, H. Jégou, and T. Mikolov, “FastText.zip: Compressing text classification models,” Dec. 12, 2016, *arXiv*: arXiv:1612.03651. doi: 10.48550/arXiv.1612.03651.
- [29] C. Tulu, “Experimental Comparison of Pre-Trained Word Embedding Vectors of Word2Vec, Glove, FastText for Word Level Semantic Text Similarity Measurement in Turkish,” *Adv. Sci. Technol. Res. J.*, vol. 16, no. 4, pp. 147–156, Oct. 2022, doi: 10.12913/22998624/152453.
- [30] M. S. I. Malik, A. Nawaz, M. M. Jamjoom, and D. I. Ignatov, “Effectiveness of ELMo embeddings, and semantic models in predicting review helpfulness,” *Intell. Data Anal.*, vol. 28, no. 4, pp. 1045–1065, Jul. 2024, doi: 10.3233/IDA-230349.
- [31] V. Zhelezniak, A. Savkov, A. Shen, and N. Hammerla, “Correlation Coefficients and Semantic Textual Similarity,” in *Proceedings of the 2019 Conference of the North*, Minneapolis, Minnesota: Association for Computational Linguistics, 2019, pp. 951–962. doi: 10.18653/v1/N19-1100.
- [32] T. Kim, K. M. Yoo, and S. Lee, “Self-Guided Contrastive Learning for BERT Sentence Representations,” in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Online: Association for Computational Linguistics, 2021, pp. 2528–2540. doi: 10.18653/v1/2021.acl-long.197.
- [33] S. M. Mohammed, K. Jacksi, and S. R. M. Zeebaree, “A state-of-the-art survey on semantic similarity for document clustering using GloVe and density-based algorithms,” *Indones. J. Electr. Eng. Comput. Sci.*, vol. 22, no. 1, p. 552, Apr. 2021, doi: 10.11591/ijeecs.v22.i1.pp552-562.

- [34] E. Hindocha, V. Yazhiny, A. Arunkumar, and P. Boobalan, “Short-text Semantic Similarity using GloVe word embedding,” vol. 06, no. 04, 2019.
- [35] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” May 24, 2019, *arXiv*: arXiv:1810.04805. doi: 10.48550/arXiv.1810.04805.
- [36] Y. Liu *et al.*, “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” Jul. 26, 2019, *arXiv*: arXiv:1907.11692. doi: 10.48550/arXiv.1907.11692.
- [37] I. Rep and V. Čeperić, “Boosting the Performance of Transformer Architectures for Semantic Textual Similarity,” Jun. 01, 2023, *arXiv*: arXiv:2306.00708. doi: 10.48550/arXiv.2306.00708.
- [38] S. Abbasi, M. R. Modarres, and M. T. Pilehvar, “NormXLogit: The Head-on-Top Never Lies”.
- [39] M. Kowsher *et al.*, “FlowNIB: An Information Bottleneck Analysis of Bidirectional vs. Unidirectional Language Models,” Oct. 09, 2025, *arXiv*: arXiv:2506.00859. doi: 10.48550/arXiv.2506.00859.
- [40] X. Huang, Y. Wu, D. Zhang, J. Hu, and Y. Long, “Improving Academic Skills Assessment with NLP and Ensemble Learning,” Oct. 13, 2024, *arXiv*: arXiv:2409.19013. doi: 10.48550/arXiv.2409.19013.
- [41] A. B. Yahia, I. Kadir, A. Abdallaoui, and A. El-Hmaidi, “Enhancing relative humidity modelling using L2 regularization updates,” *Sci. Rep.*, vol. 15, no. 1, p. 14796, Apr. 2025, doi: 10.1038/s41598-025-94356-9.
- [42] K. Ethayarajh, “How Contextual are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings,” Sep. 02, 2019, *arXiv*: arXiv:1909.00512. doi: 10.48550/arXiv.1909.00512.
- [43] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Hong Kong, China: Association for Computational Linguistics, 2019, pp. 3980–3990. doi: 10.18653/v1/D19-1410.
- [44] D. Cer *et al.*, “Universal Sentence Encoder,” Apr. 12, 2018, *arXiv*: arXiv:1803.11175. doi: 10.48550/arXiv.1803.11175.
- [45] H. Aldarmaki and M. Diab, “Context-Aware Cross-Lingual Mapping,” Mar. 31, 2019, *arXiv*: arXiv:1903.03243. doi: 10.48550/arXiv.1903.03243.
- [46] S. K. Gaddipati, D. Nair, and P. G. Plöger, “Comparative Evaluation of Pretrained Transfer Learning Models on Automatic Short Answer Grading,” Sep. 02, 2020, *arXiv*: arXiv:2009.01303. doi: 10.48550/arXiv.2009.01303.
- [47] D. Lalita Luhurkinanti, P. Dewi Purnamasari, T. Tsunakawa, and A. Agung Putri Ratna, “Japanese Short Answer Grading for Japanese Language Learners Using the Contextual Representation of BERT,” *IEEE Access*, vol. 13, pp. 17195–17207, 2025,

- doi: 10.1109/ACCESS.2025.3532659.
- [48] D. Cer, M. Diab, E. Agirre, I. Lopez-Gazpio, and L. Specia, “SemEval-2017 Task 1: Semantic Textual Similarity - Multilingual and Cross-lingual Focused Evaluation,” in *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, 2017, pp. 1–14. doi: 10.18653/v1/S17-2001.
 - [49] N. Reimers and I. Gurevych, “Semantic Textual Similarity,” Sentence Transformers Documentation. [Online]. Available: https://sbert.net/examples/sentence_transformer/training/sts/README.html
 - [50] A. I. Nikolaichuk, “Semantic textual similarity models evaluation for tasks requiring binary threshold decisions,” Kharkiv National University of Radio Electronics, Department of Systems Engineering, Kharkiv, Ukraine, pp. 470–472.
 - [51] B. Kriuk and F. Kriuk, “Multi-Objective Optimal Threshold Selection for Similarity Functions in Siamese Networks for Semantic Textual Similarity Tasks,” Jul. 01, 2024, *Computer Science and Mathematics*. doi: 10.20944/preprints202407.0020.v1.
 - [52] D. Theron, “Contextualizing the Limits of Model & Evaluation Dataset Curation on Semantic Similarity Classification Tasks”.
 - [53] I. Atoum, “Scaled Pearson’s Correlation Coefficient for Evaluating Text Similarity Measures,” *Mod. Appl. Sci.*, vol. 13, no. 10, p. 26, Sep. 2019, doi: 10.5539/mas.v13n10p26.
 - [54] P. Schober, C. Boer, and L. A. Schwarte, “Correlation Coefficients: Appropriate Use and Interpretation,” *Anesth. Analg.*, vol. 126, no. 5, pp. 1763–1768, May 2018, doi: 10.1213/ANE.0000000000002864.
 - [55] V. Rawte, A. Gupta, and M. J. Zaki, “A Comparative Analysis of Temporal Long Text Similarity: Application to Financial Documents,” in *Mining Data for Financial Applications*, vol. 12591, V. Bitetta, I. Bordino, A. Ferretti, F. Gullo, G. Ponti, and L. Severini, Eds., in Lecture Notes in Computer Science, vol. 12591. , Cham: Springer International Publishing, 2021, pp. 77–91. doi: 10.1007/978-3-030-66981-2_7.
 - [56] C. Willmott and K. Matsuura, “Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance,” *Clim. Res.*, vol. 30, pp. 79–82, 2005, doi: 10.3354/cr030079.
 - [57] T. O. Hodson, “Root-mean-square error (RMSE) or mean absolute error (MAE): when to use them or not,” *Geosci. Model Dev.*, vol. 15, no. 14, pp. 5481–5487, Jul. 2022, doi: 10.5194/gmd-15-5481-2022.
 - [58] C. Manning, P. Raghavan, and H. Schuetze, “Introduction to Information Retrieval,” 2009.
 - [59] OpenAI, *ChatGPT (GPT-5.1), Large Language Model. 2025*. [Online]. Available: <https://chatgpt.com>

APPENDIX

A. IMPLEMENTATION CODE SNIPPETS AND RESULTS

A.1 Text Preprocessing

Preprocess Code

- Lowercasing
- Tokenization
- Stopword and punctuation removal
- Lemmatization.

```
# Preprocess sentences by tokenizing, removing stopwords and punctuation, and
# applying lemmatization.
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

def preprocess(text: str) -> str:
    tokens = word_tokenize(text.lower())
    tokens = [w for w in tokens if w not in stop_words and w not in
string.punctuation]
    return " ".join(lemmatizer.lemmatize(w) for w in tokens)

def apply_preprocess(df):
    s1_pre = df["sentence1"].apply(preprocess)
    s2_pre = df["sentence2"].apply(preprocess)
    return (
        s1_pre.tolist(), s2_pre.tolist(),
        df["sentence1"].tolist(), df["sentence2"].tolist(),
        df["score"].astype(float).to_numpy()
    )
s1_pre_tr, s2_pre_tr, s1_raw_tr, s2_raw_tr, y_train = apply_preprocess(df_train)
s1_pre_de, s2_pre_de, s1_raw_de, s2_raw_de, y_dev = apply_preprocess(df_dev)
s1_pre_te, s2_pre_te, s1_raw_te, s2_raw_te, y_test = apply_preprocess(df_test)
```

Figure 30 Code Snippet for Preprocessing

Output:

	Raw Sentence 1	Raw Sentence 2	Preprocessed Sentence 1	Preprocessed Sentence 2
0	A plane is taking off.	An air plane is taking off.	plane taking	air plane taking
1	A man is playing a large flute.	A man is playing a flute.	man playing large flute	man playing flute
2	A man is spreading shredded cheese on a pizza.	A man is spreading shredded cheese on an uncooked pizza.	man spreading shredded cheese pizza	man spreading shredded cheese uncooked pizza
3	Three men are playing chess.	Two men are playing chess.	three men playing chess	two men playing chess
4	A man is playing the cello.	A man seated is playing the cello.	man playing cello	man seated playing cello

Figure 9 Preprocessed Dataset Overview

A.2 Ridge Regression Model Creation

```
# Define a Ridge Regression model with L2 regularization for similarity prediction.
def make_ridge_model(input_dim=3, lambda_l2=0.01, use_bias=True, lr=1e-2):
    inp = layers.Input(shape=(input_dim,), name="features")
    out = layers.Dense(
        1,
        activation="linear",
        use_bias=use_bias,
        kernel_regularizer=regularizers.l2(lambda_l2),
        name="ridge_head",
    )(inp)
    model = models.Model(inp, out, name="ridge_similarity")
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=lr),
        loss="mse",
        metrics=["mae"]
    )
    return model
```

Figure 20 Code Snippet for Training a Ridge Regression Model

A.3 Hyperparameter Setting Code Snippet

```
# hyperparameters
k_values = [8, 9, 10]
lambda_candidates = [1e-4, 3e-4, 1e-3, 3e-3, 1e-2, 3e-2, 1e-1]
epoch_candidates = [5, 10, 20]
patience_candidates = [2, 5, 8]
batch_size = 64
learning_rate = 1e-2
thr_grid_steps = 2001
TRUE_SIM_THRESHOLD = globals().get("TRUE_SIM_THRESHOLD", 0.5)
```

Figure 31 Code Snippet for Hyperparameter Setting

Table IX Hyperparameter Search Space per Training Settings

Setting	K values	Epochs	Patience	Lambda candidates	Batch Size	Learning rate	Threshold
S1	{2, 3, 4}	{50, 100, 200}	{5, 10, 20}	{1e-4, 3e-4, 1e-3, 3e-3, 1e-2, 3e-2, 1e-1}	64	1e-2	0.5
S2	{2, 3, 4}	{5, 10, 20}	{2, 5, 8}	{1e-4, 3e-4, 1e-3, 3e-3, 1e-2, 3e-2, 1e-1}	64	1e-2	0.5
S3	{5, 6, 7}	{5, 10, 20}	{2, 5, 8}	{1e-4, 3e-4, 1e-3, 3e-3, 1e-2, 3e-2, 1e-1}	64	1e-2	0.5
S4	{8, 9, 10}	{5, 10, 20}	{2, 5, 8}	{1e-4, 3e-4, 1e-3, 3e-3, 1e-2, 3e-2, 1e-1}	64	1e-2	0.5

Table X Best-Performing Hyperparameter Configuration

Setting	Epochs	Best Epoch	Patience	K value	Best Lambda	Validation Mae	Validation Loss
S4	10	10	8	8	1e-4	0.1499	0.0360
S3	20	20	2	5	3e-4	0.1480	0.0356
S1	200	6	5	3	1e-2	0.1486	0.0390
S2	10	5	2	2	1e-2	0.1486	0.0390
S3	10	4	5	5	1e-2	0.1483	0.0392
S2	20	9	8	3	1e-2	0.1486	0.0389
S1	50	20	5	3	3e-3	0.1473	0.0362
S4	20	20	5	8	1e-2	0.1486	0.0390
S3	20	14	8	5	1e-2	0.1483	0.0392
S2	20	20	5	2	3e-4	0.1506	0.0362

A.4 Code Snippets for the Training Experiment

```
# training experiment
exp_counter = 0
per_k_aggregates = {k: [] for k in k_values}

for k_splits in k_values:
    print(f"\nStarting k={k_splits}")
    kfold = KFold(n_splits=k_splits, shuffle=True, random_state=42)

    for epochs, patience in itertools.product(epoch_candidates, patience_candidates):
        print(f"\ncombo: epochs={epochs}, patience={patience}")

        # 1) CV over lambda
        lambda_summary = []
        for lam in lambda_candidates:
            fold_pccs, fold_maes, fold_mses = [], [], []
            for fold_idx, (tr_idx, va_idx) in enumerate(kfold.split(X_train)):
                X_tr, X_va = X_train[tr_idx], X_train[va_idx]
                y_tr, y_va = Y_train[tr_idx], Y_train[va_idx]

                model_i = make_ridge_model(input_dim=X_tr.shape[1], lambda_l2=lam,
lr=learning_rate)
                es_cv = callbacks.EarlyStopping(monitor="val_loss",
patience=patience, restore_best_weights=True)

                hist = model_i.fit(
                    X_tr, y_tr,
                    validation_data=(X_va, y_va),
                    epochs=epochs, batch_size=batch_size,
                    callbacks=[es_cv], verbose=0
                )

                # fold best values
                val_loss = hist.history.get("val_loss", [])
                best_epoch = int(np.argmin(val_loss)) + 1 if len(val_loss) > 0 else 0
                best_loss = float(np.min(val_loss)) if len(val_loss) > 0 else np.nan

                # predictions on val
                y_pred_va = model_i.predict(X_va, verbose=0).ravel()
                pcc, _ = safe_pearsonr(y_va, y_pred_va)
                mae = float(mean_absolute_error(y_va, y_pred_va))
                mse = float(mean_squared_error(y_va, y_pred_va))

                fold_pccs.append(pcc); fold_maes.append(mae); fold_mses.append(mse)

            # append fold row (exp_id blank for now)
            with open(folds_csv, "a", newline="", encoding="utf-8") as f:
                csv.writer(f).writerow([
                    "", k_splits, lam, epochs, patience,
                    fold_idx, pcc, mae, mse,
                    best_epoch, best_loss
                ])

        lambda_summary.append({
            "lambda": lam,
            "mean_pcc": float(np.mean(fold_pccs)),
            "std_pcc": float(np.std(fold_pccs)),
            "mean_mae": float(np.mean(fold_maes)),
            "std_mae": float(np.std(fold_maes)),
            "mean_mse": float(np.mean(fold_mses)),
            "std_mse": float(np.std(fold_mses))
        })

    # choose best lambda
    best_lambda_row = max(lambda_summary, key=lambda x: x["mean_pcc"])
    best_lambda = best_lambda_row["lambda"]
```

```

# 2) retrain on TRAIN+DEV
X_full = np.concatenate([X_train, X_dev])
y_full = np.concatenate([Y_train, Y_dev])

model_final = make_ridge_model(input_dim=X_full.shape[1],
lambda_l2=best_lambda, lr=learning_rate)
es_final = callbacks.EarlyStopping(monitor="val_loss", patience=patience,
restore_best_weights=True)

hist_final = model_final.fit(
    X_full, y_full,
    epochs=epochs, batch_size=batch_size, validation_split=0.1,
    callbacks=[es_final], verbose=0
)

# epoch log
history = getattr(hist_final, "history", {})
out_epoch_path = os.path.join(EPOCH_LOG_DIR,
f"tgd(e={epochs},p={patience},k={k_splits}).epochs_log.csv")
if history:
    # long-format rows
    first_key = next(iter(history.keys()))
    num_epochs_recorded = len(history.get(first_key, []))
    rows = []
    for ep in range(num_epochs_recorded):
        r = {
            "epoch": ep + 1,
            "k_splits": k_splits,
            "epochs_setting": epochs,

            "patience_setting": patience,
            "best_lambda": best_lambda
        }
        for key, vals in history.items():
            r[key] = float(vals[ep]) if ep < len(vals) else np.nan
        rows.append(r)
    # overwrite per-exp epoch-log csv with real rows
    pd.DataFrame(rows).to_csv(out_epoch_path, index=False, encoding="utf-8")
    print(f"Saved epoch log -> {out_epoch_path} (epochs recorded:
(num_epochs_recorded))")
else:
    if not os.path.exists(out_epoch_path):
pd.DataFrame([{"epoch":1,"k_splits":k_splits,"epochs_setting":epochs,"patience_settin
g":patience}])
        .head(0).to_csv(out_epoch_path, index=False)
        print(f"Warning: empty history for e={epochs},p={patience},k={k_splits} -
created/left file {out_epoch_path}")

# predictions & metrics
dev_pred = model_final.predict(X_dev, verbose=0).ravel()
test_pred = model_final.predict(X_test, verbose=0).ravel()
dev_metrics = compute_regression_metrics(Y_dev, dev_pred)
test_metrics = compute_regression_metrics(Y_test, test_pred)

# tune threshold & compute binary metrics
y_dev_bin = (Y_dev >= TRUE_SIM_THRESHOLD).astype(int)
pred_thr, dev_best_f1 = tune_threshold_on_dev(dev_pred, y_dev_bin,
search_min=0.0, search_max=1.0, steps=thr_grid_steps)
bin_metrics = compute_binary_metrics_with_thr(Y_test, test_pred, pred_thr,
true_sim_threshold=TRUE_SIM_THRESHOLD)

# save model (.h5)
model_name = f"tgd(ensemble,e={epochs},p={patience},k={k_splits}).h5"
model_path = os.path.join(MODELS_DIR, model_name)
try:
    model_final.save(model_path)
except Exception as e:
    print("WARN: model save failed:", e)

```

```

        model_path = ""

    # produce & save plot
    try:
        hist = getattr(hist_final, "history", {})
        loss = hist.get("loss", [])
        val_loss = hist.get("val_loss", [])
        mae_tr = hist.get("mae", []) or hist.get("mean_absolute_error", [])
        mae_val = hist.get("val_mae", []) or hist.get("val_mean_absolute_error",
    [])

        plot_name = f"plot_tgd(e{epochs}_p{patience}_k{k_splits}).png"
        plot_path = os.path.join(PLOTS_DIR, plot_name)

        plt.figure(figsize=(12,5))
        plt.subplot(1,2,1)
        if len(loss): plt.plot(loss, label="Train Loss (MSE)")
        if len(val_loss): plt.plot(val_loss, label="Val Loss (MSE)")
        plt.title("Training vs Validation Loss"); plt.xlabel("Epoch");
    plt.ylabel("MSE"); plt.legend()
        plt.subplot(1,2,2)
        if len(mae_tr): plt.plot(mae_tr, label="Train MAE")
        if len(mae_val): plt.plot(mae_val, label="Val MAE")
        plt.title("Training vs Validation MAE"); plt.xlabel("Epoch");
    plt.ylabel("MAE"); plt.legend()
        plt.tight_layout(); plt.savefig(plot_path); plt.close()
    except Exception as e:
        print("WARN: plot save failed:", e)
        plot_path = ""

    # write experiment row to experiments_summary.csv
    exp_id = str(uuid.uuid4())[:8]
    with open(experiments_csv, "a", newline="", encoding="utf-8") as f:
        csv.writer(f).writerow([
            exp_id, k_splits, best_lambda, epochs, patience,
            best_lambda_row["mean_pcc"], best_lambda_row["std_pcc"],
            best_lambda_row["mean_mae"], best_lambda_row["std_mae"],
            best_lambda_row["mean_mse"], best_lambda_row["std_mse"],
            best_lambda,
            dev_metrics["pcc"], test_metrics["pcc"], test_metrics["mse"],
    test_metrics["mae"],
            pred_thr, dev_best_f1, TRUE_SIM_THRESHOLD,
            bin_metrics["accuracy"], bin_metrics["precision"],
            bin_metrics["recall"], bin_metrics["f1"],
            model_path, plot_path
        ])

    # append to per-k aggregates
    per_k_aggregates[k_splits].append({
        "exp_id": exp_id,
        "epochs": epochs,
        "patience": patience,
        "best_lambda": best_lambda,
        "val_mean_pcc": best_lambda_row["mean_pcc"],
        "val_std_pcc": best_lambda_row["std_pcc"],
        "val_mean_mae": best_lambda_row["mean_mae"],
        "val_std_mae": best_lambda_row["std_mae"],
        "val_mean_mse": best_lambda_row["mean_mse"],
        "val_std_mse": best_lambda_row["std_mse"],
        "dev_pcc": dev_metrics["pcc"],
        "test_pcc": test_metrics["pcc"],
        "test_mse": test_metrics["mse"],
        "test_mae": test_metrics["mae"],
        "pred_thr": pred_thr,
        "dev_best_f1": dev_best_f1,
        "test_accuracy": bin_metrics["accuracy"],
        "test_precision": bin_metrics["precision"],
        "test_recall": bin_metrics["recall"],
        "test_f1": bin_metrics["f1"],
        "model_path": model_path,
        "plot_path": plot_path
    })

```

```

    })

    exp_counter += 1
    print(f"[{exp_counter}] k={k_splits} e={epochs} p={patience}")
    bestλ={best_lambda:.0e}
    test_pcc={test_metrics['pcc']:.4f} pred_thr={pred_thr:.4f}
    test_f1={bin_metrics['f1']:.4f}")

print("\nALL experiments finished.")

for k, rows in per_k_aggregates.items():
    df = pd.DataFrame(rows)
    outp_k = os.path.join(PER_K_DIR, f"tgd_per_k{k}.csv")
    dfk.to_csv(outp_k, index=False, encoding="utf-8")
    print("Saved per-k CSV:", outp_k)

print("Done.")

```

Figure 32 Code Snippets for the Training Experiment

Output:

Sample illustration below of training data was saved to a CSV file.

```

Starting k=8

combo: epochs=5, patience=2

Saved epoch log ->
/content/drive/MyDrive/(k=8,9,10)TGD_TRAIN_HISTORY/tgd_epochs_log/tgd(e=5,p=2,k=8).ep
ochs_log.csv (epochs recorded: 5)

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or
`keras.saving.save_model(model, 'my_model.keras')`.

[1] k=8 e=5 p=2 bestλ=1e-01 test_pcc=0.6655 pred_thr=0.4680 test_f1=0.8056

combo: epochs=5, patience=5

Saved epoch log ->
/content/drive/MyDrive/(k=8,9,10)TGD_TRAIN_HISTORY/tgd_epochs_log/tgd(e=5,p=5,k=8).ep
ochs_log.csv (epochs recorded: 5)

```

Figure 33 Sample Output of Training Data Stored as CSV

A.5 Code Snippet for Semantic Similarity Binarization and Threshold Tuning

```
# Binarization of semantic similarity scores.
# Continuous similarity scores from the STS dataset are converted into binary labels
# using a threshold (GOLD_THR = 0.5). Scores  $\geq 0.5$  are labeled as semantically
similar (1),
# while scores  $< 0.5$  are labeled as dissimilar (0). This transformation allows the
# semantic similarity task to be evaluated as a binary classification problem,
# enabling the computation of metrics such as accuracy and F1-score.

GOLD_THR = globals().get("TRUE_SIM_THRESHOLD", 0.5)    grid_steps = 2001

def tune_threshold_on_dev(dev_scores, y_dev_bin, steps=grid_steps, search_min=0.0,
search_max=1.0):
    grid = np.linspace(search_min, search_max, steps)
    best_f1 = -1.0
    best_thr = grid[0]
    for thr in grid:
        pred = (dev_scores >= thr).astype(int)
        _, _, f1, _ = precision_recall_fscore_support(y_dev_bin, pred,
average="binary", zero_division=0)
        if f1 > best_f1:
            best_f1 = f1
            best_thr = thr
    return float(best_thr), float(best_f1)

def classification_with_thr(y_test_cont, y_test_bin, pred_thr):
    y_pred_bin = (y_test_cont >= pred_thr).astype(int)
    acc = accuracy_score(y_test_bin, y_pred_bin)
    prec, rec, f1, _ = precision_recall_fscore_support(y_test_bin, y_pred_bin,
average="binary", zero_division=0)
    cm = confusion_matrix(y_test_bin, y_pred_bin, labels=[0,1])
    return float(acc), float(prec), float(rec), float(f1), cm

# Prepare gold binary labels
y_dev_bin = (Y_dev >= GOLD_THR).astype(int)
y_test_bin = (Y_test >= GOLD_THR).astype(int)

# compute for each standalone model
rows = []
json_rows = []

for model_name, (dev_name, test_name) in required.items():
    dev_scores = globals()[dev_name].astype(float)
    test_scores = globals()[test_name].astype(float)

    # tune threshold on dev
    pred_thr, best_f1_dev = tune_threshold_on_dev(dev_scores, y_dev_bin,
steps=grid_steps, search_min=0.0, search_max=1.0)

    # metrics on test using tuned threshold
    acc, prec, rec, f1, cm = classification_with_thr(test_scores, y_test_bin,
pred_thr)

    row = {
        "model": model_name,
        "Pred thr": pred_thr,
        "Dev best_f1": best_f1_dev,
        "test_pcc": pcc,
        "test_accuracy": acc,
        "test_f1": f1
    }
    rows.append(row)

# Save outputs
df_out = pd.DataFrame(rows)
df_out.to_csv(out_csv, index=False)
```

```

with open(out_json, "w", encoding="utf-8") as jf:
    json.dump(json_rows, jf, indent=2)

print("Standalone tuned metrics saved:")
print(" - CSV:", out_csv)
print(" - JSON:", out_json)
display(df_out)
# Print confusion matrices
print("\nConfusion matrices (format [[TN, FP],[FN, TP]])")
for jr in json_rows:
    print(f"\n{jr['model']} (pred_thr={jr['Pred thr']:.4f})")
    print(pd.DataFrame(jr["confusion_matrix"]))

```

Figure 34 Code Snippet for Semantic Similarity Binarization and Threshold Tuning

Output:

```

Confusion matrices (format [[TN, FP],[FN, TP]])

TF-IDF (pred_thr=0.2500)
    0    1
0 282 325
1  47 725

GloVe (pred_thr=0.7480)
    0    1
0 313 294
1  84 688

DeBERTa (pred_thr=0.9290)
    0    1
0 143 464
1 112 660

```

Figure 35 Confusion Matrix

A.6 TGD System Dashboard

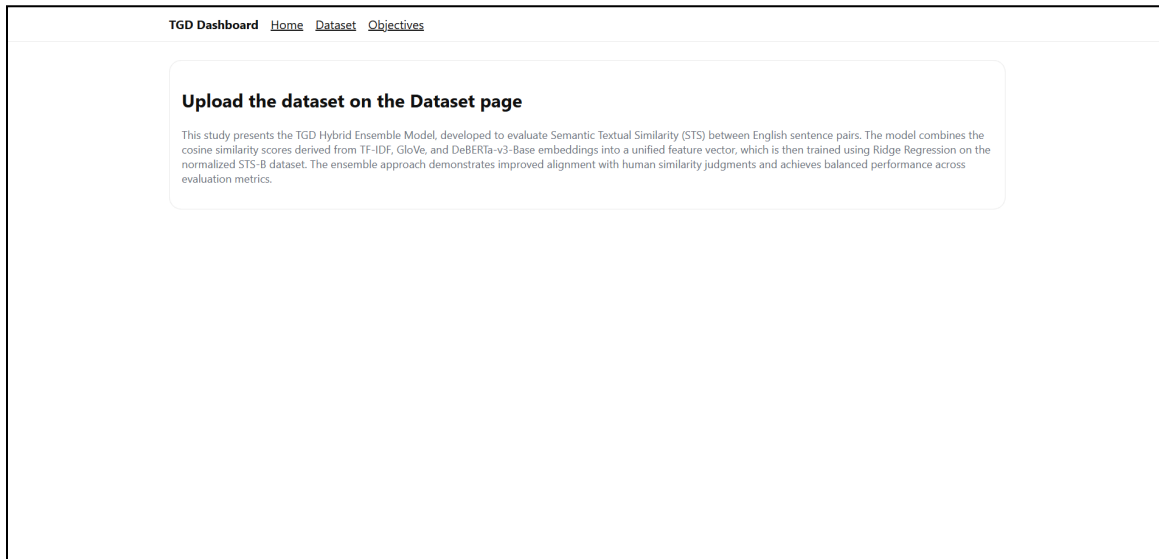


Figure 36 TGD Dashboard Homepage

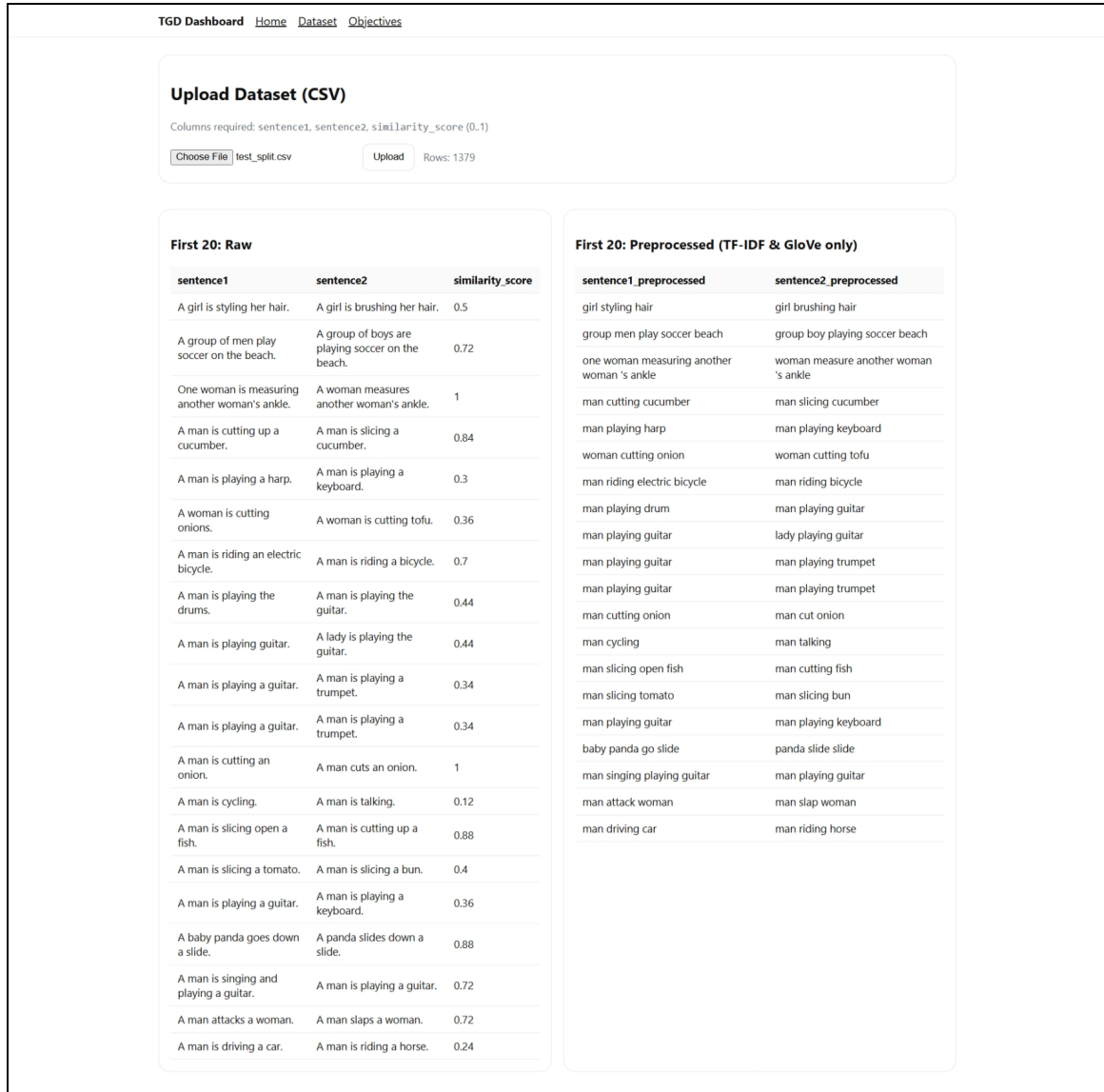


Figure 37 TGD Dashboard Dataset Page

Expected Output: The test dataset CSV is successfully uploaded and preprocessed.

TGD Dashboard

[Home](#)

[Dataset](#)

[Objectives](#)

Objectives of the Study

1) Generate Embeddings

2) Compute Cosine Similarity Scores

3) Form a Feature Vector

4) Train History

5) Evaluate

1. Embeddings Preview

TF-IDF (top terms)			GloVe (first 10 dims)				DeBERTa (first 10 dims)					
No.	Top TF-IDF Terms (S1)	Top TF-IDF Terms (S2)	No.	S1 Vector (first 10)	S2 Vector (first 10)	Sentence 1 (proc)	Sentence 2 (proc)	No.	S1 Vector (first 10)	S2 Vector (first 10)	Sentence 1	Sentence 2
1	styling (0.7317), hair (0.5652), girl (0.3809)	brushing (0.7059), hair (0.5874), girl (0.3959)	1	0.0004, -0.1115, -0.3969, -0.1707, -0.1546, -0.0412, -0.3012, 0.0975, 0.1233, -0.9641,	-0.1481, -0.0525, -0.3660, -0.0310, -0.0699, -0.2693, -0.3413, 0.0081, 0.3452, -0.8179,	girl styling hair	girl brushing hair	1	0.1776, 0.0729, -0.0906, 0.2049, -0.1703, -0.3401, 0.1908, 0.0016, -0.0304, -0.1389,	0.1032, 0.0144, -0.0838, 0.1594, -0.1521, -0.3513, 0.1420, 0.0158, -0.0112, -0.1178,	A girl is styling her hair.	A girl is brushing her hair.
2	soccer (0.5118), beach (0.4549), play (0.4378), men (0.4217)	soccer (0.5465), beach (0.4857), group (0.4294), boy (0.4047)										
3	woman (0.6687), another (0.5612), one (0.4877)	measure (0.6133), woman (0.6050), another (0.5077)										
	cucumber (0.7609), cutting (0.7735), slicing (0.7735)	cucumber (0.7735), slicing (0.7735)										

Figure 38 TGD Dashboard Objective Page - Objective 1: Generate Embeddings
Expected Output: Generated sentence embeddings.

TGD Dashboard Home Dataset Objectives						
2. Cosine Similarity (preview)						
Download cosines CSV						
sentence1	sentence2	sim_deberta	sim_glove	sim_tfidf	similarity_score	
A girl is styling her hair.	A girl is brushing her hair.	0.9923	0.809	0.4828	0.5	
A group of men play soccer on the beach.	A group of boys are playing soccer on the beach.	0.9919	0.9143	0.6733	0.72	
One woman is measuring another woman's ankle.	A woman measures another woman's ankle.	0.9807	0.958	0.6895	1	
A man is cutting up a cucumber.	A man is slicing a cucumber.	0.9893	0.8807	0.6943	0.84	
A man is playing a harp.	A man is playing a keyboard.	0.9944	0.806	0.3281	0.3	
A woman is cutting onions.	A woman is cutting tofu.	0.9909	0.7813	0.4655	0.36	
A man is riding an electric bicycle.	A man is riding a bicycle.	0.989	0.9198	0.7482	0.7	
A man is playing the drums.	A man is playing the guitar.	0.9931	0.881	0.4067	0.44	
A man is playing guitar.	A lady is playing the guitar.	0.9896	0.8686	0.6268	0.44	
A man is playing a guitar.	A man is playing a trumpet.	0.9963	0.907	0.4067	0.34	

Figure 39 TGD Dashboard Objective Page - Objective 2: Compute Cosine Similarities
Expected Output: Cosine similarity scores are computed for each sentence pair.

TGD Dashboard Home Dataset Objectives			
3. Feature Vector (x1, x2, x3)			
Download features CSV			
No.	x1 = TF-IDF	x2 = GloVe	x3 = DeBERTa
1	0.4828	0.809	0.9923
2	0.6733	0.9143	0.9919
3	0.6895	0.958	0.9807
4	0.6943	0.8807	0.9893
5	0.3281	0.806	0.9944
6	0.4655	0.7813	0.9909
7	0.7482	0.9198	0.989
8	0.4067	0.881	0.9931
9	0.6268	0.8686	0.9896
10	0.4067	0.907	0.9963

Figure 40 TGD Dashboard Objective Page - Objective 3: Form a Feature Vector

Expected Output: A unified feature vector is formed from the generated embeddings and similarity scores.

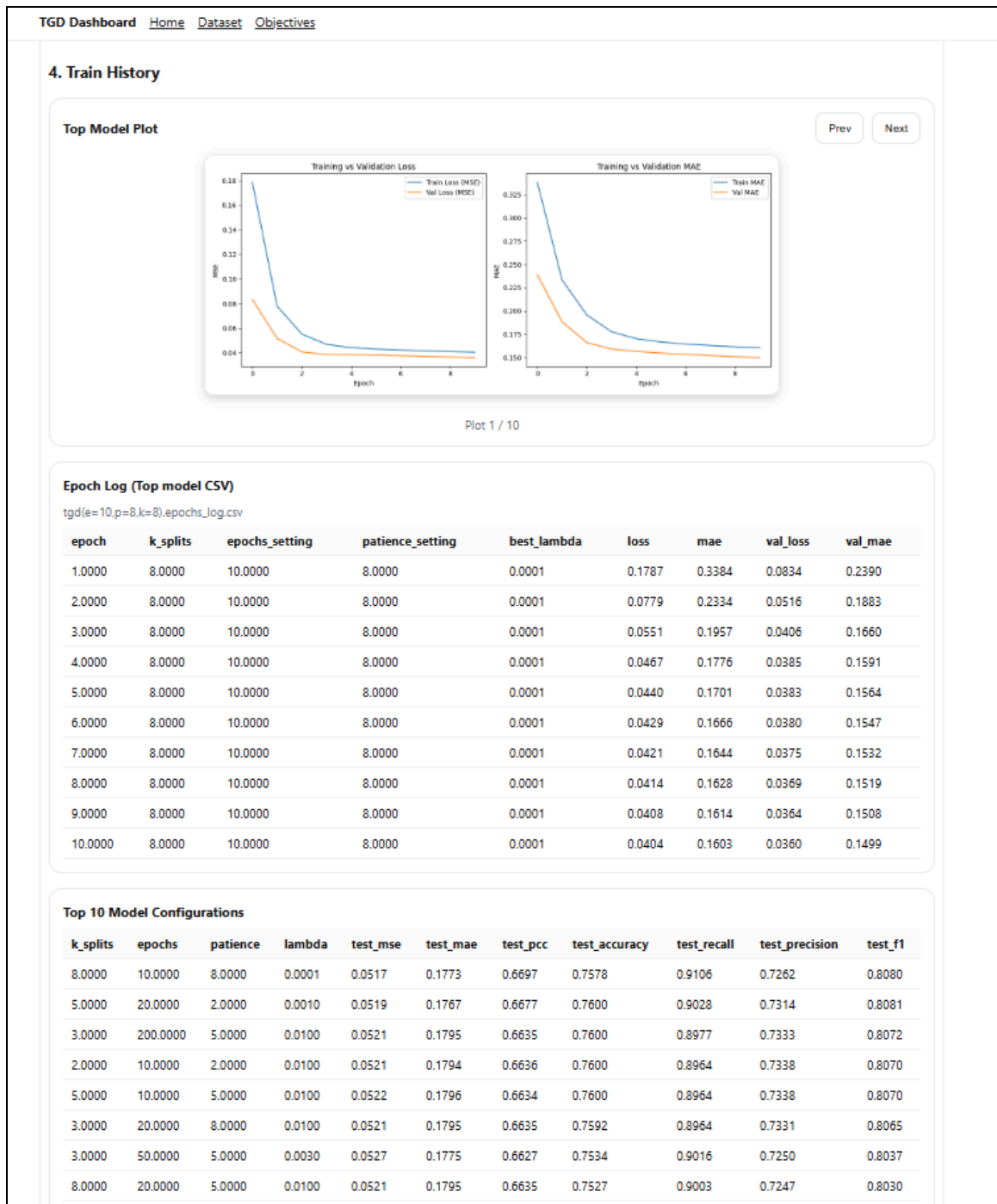


Figure 41 TGD Dashboard Objective Page - Objective 4: Train History

Expected Output: Training history and learning progress of the model are displayed.

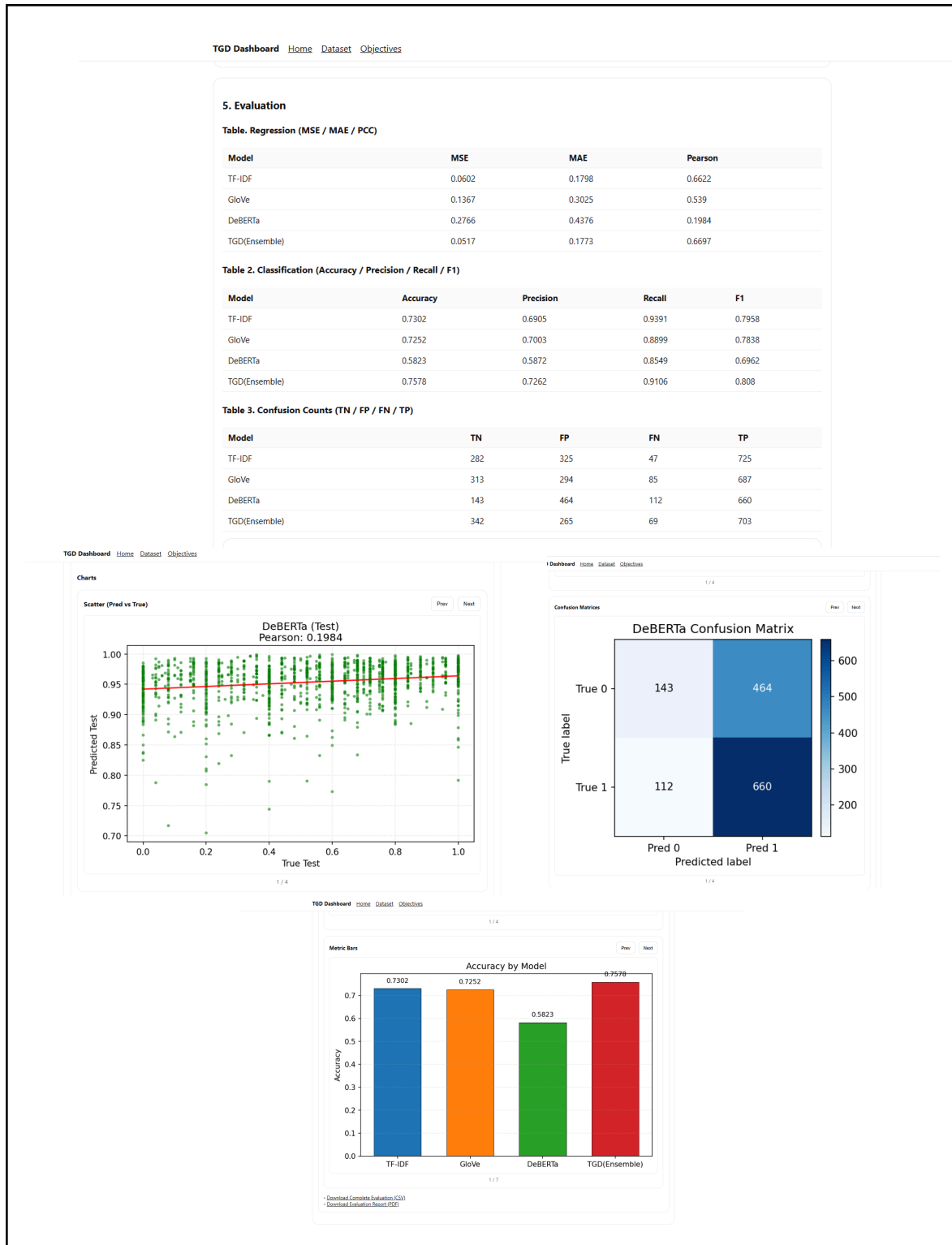
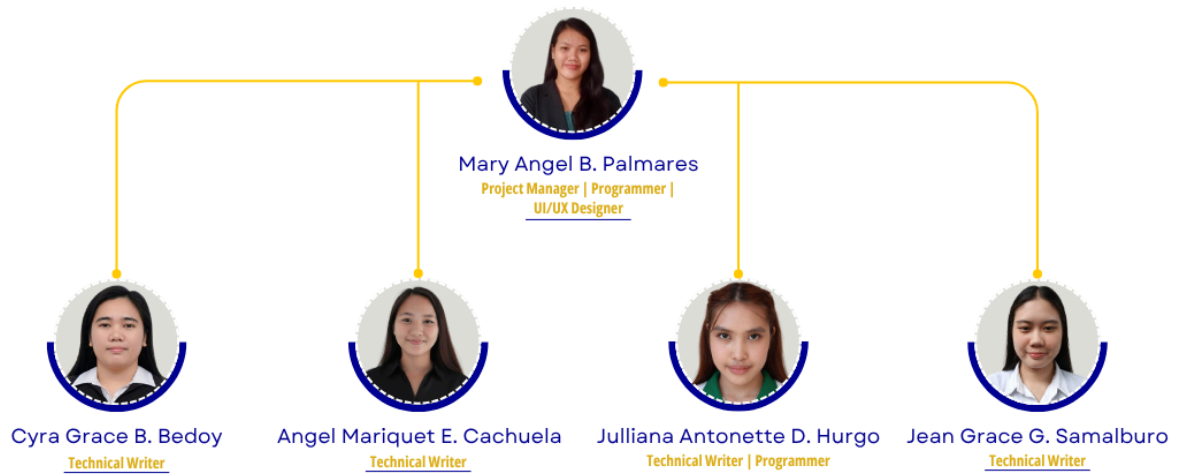


Figure 42 TGD Dashboard Objective Page - Objective 5: Evaluate

Expected Output: Evaluation results and performance metrics of the proposed model are displayed.

B. ORGANIZATIONAL CHART



Description of Roles

Mary Angel B. Palmares – Project Manager / Programmer / UI-UX Designer

Mary Angel B. Palmares serves as the Project Manager, overseeing the overall development and implementation of the study. She ensures that the objectives of the proposed weighted ensemble model are achieved within the specified timeline by coordinating team tasks and monitoring progress. She is responsible for a significant portion of the programming tasks and contributes to the implementation of the proposed model. She also designs the dashboard, ensuring that the presentation of results is clear, functional, and user-friendly.

Cyra Grace B. Bedoy – Technical Writer

Cyra Grace B. Bedoy provides support in the documentation process by assisting in gathering related literature, proofreading, and ensuring that the content is clear and properly formatted. She contributes to the organization of references and helps verify the

accuracy of cited materials. Additionally, she reviews sections of the paper to improve readability and ensure alignment with academic writing standards.

Angel Mariqueta E. Cachuela – Technical Writer

Angel Mariquet E. Cachuela contributes to the documentation of the study by assisting in writing, organizing, and refining sections of the research paper. She ensures that the content is clear, coherent, and aligned with academic standards. She is also responsible for coordinating with significant persons to obtain required signatures for the necessary research documents.

Julliana Antonette D. Hurgo – Technical Writer / Programmer

Julliana Antonette D. Hurgo is responsible for both documentation and programming tasks. She contributes extensively to the research paper, particularly in technical sections such as the methodology, implementation, and analysis. In addition, she implemented core functionalities of the proposed model, ensuring that the technical components are accurate and aligned with the study's objectives.

Jean Grace G. Samalburo – Technical Writer

Jean Grace G. Samalburo supports the documentation of the study by assisting in literature sourcing, proofreading, and formatting. She helps in organizing the structure of the research paper and ensures that formatting guidelines are consistently followed. She also reviews the content to enhance clarity and coherence, contributing to the overall refinement of the document.

C. RESEARCHER'S PROFILE

CURRICULUM VITAE

CYRA GRACE B. BEDOY

Home Address: San Florentino, San Rafael, Iloilo 5039

Contact No.: +639157356545

Email: cgbbedoy103003@gmail.com



PERSONAL INFORMATION

Date of Birth	:	October 30, 2003
Place of Birth	:	WVMC, Mandurriao, Iloilo City, Iloilo 5000
Age	:	22
Civil Status	:	Single
Height	:	155 cm
Weight	:	85 kg
Nationality	:	Filipino
Religion Affiliation	:	Roman Catholic

FAMILY BACKGROUND

Siblings	:	Cyrus John B. Bedoy Carl Stephen B. Bedoy Cathleen Ann Marie B. Bedoy Carla May B. Bedoy
Father	:	Rogelio B. Bedoy
Mother (Maiden)	:	Nelia C. Bañas

EDUCATIONAL QUALIFICATION

<i>Level</i>	<i>School</i>	<i>Year Graduated</i>
Elementary	San Florentino Elementary School	2016
Junior High School	San Rafael National High School	2020
Senior High School	San Rafael National High School	2022
Tertiary	Iloilo Science and Technology University	2026

CURRICULUM VITAE

ANGEL MARIQUET E. CACHUELA

Home Address: Poblacion West, Oton, Iloilo 5020

Contact No.: +639566704235

Email: angelmariquetcachuela22@gmail.com



PERSONAL INFORMATION

Date of Birth	:	March 15, 2003
Place of Birth	:	WVMC, Mandurriao, Iloilo City, Iloilo 5000
Age	:	22
Civil Status	:	Single
Height	:	149 cm
Weight	:	48 kg
Nationality	:	Filipino
Religion Affiliation	:	Roman Catholic

FAMILY BACKGROUND

Siblings	:	Rowella Jane C. Celiz Roann C. Tadiaque Ace Andrue E. Cachuela Angelo Ruel E. Cachuela
Father	:	Ruel M. Cachuela
Mother (Maiden)	:	Ma. Angeli C. Enrile

EDUCATIONAL QUALIFICATION

<i>Level</i>	<i>School</i>	<i>Year Graduated</i>
Elementary	Oton Central Elementary School	2016
Junior High School	Oton National High School	2020
Senior High School	Oton National High School	2022
Tertiary	Iloilo Science and Technology University	2026

CURRICULUM VITAE

JULLIANA ANTONETTE D. HURGO

Home Address: Siniba-an, Dingle, Iloilo 5035

Contact No.: +639707989461

Email: antonettehurgo@gmail.com



PERSONAL INFORMATION

Date of Birth	:	October 10, 2003
Place of Birth	:	Palanan, Makati City, Manila 1235
Age	:	22
Civil Status	:	Single
Height	:	170.18 cm
Weight	:	55 kg
Nationality	:	Filipino
Religion Affiliation	:	Roman Catholic

FAMILY BACKGROUND

Siblings	:	Joshua D. Hurgo
Father	:	Jolibert D. Hurgo
Mother (Maiden)	:	Vilma D. De Mesa

EDUCATIONAL QUALIFICATION

<i>Level</i>	<i>School</i>	<i>Year Graduated</i>
Elementary	Siniba-an Elementary School	2016
Junior High School	Mater Carmeli School	2020
Senior High School	Mater Carmeli School	2022
Tertiary	Iloilo Science and Technology University	2026

CURRICULUM VITAE

MARY ANGEL B. PALMARES

Home Address: Poblacion Norte, Ivisan, Capiz 5805

Contact No.: +639566956831

Email: maryangel2036@gmail.com



PERSONAL INFORMATION

Date of Birth	:	August 25, 2003
Place of Birth	:	Poblacion Norte, Ivisan, Capiz 5805
Age	:	22
Civil Status	:	Single
Height	:	165.5 cm
Weight	:	58 kg
Nationality	:	Filipino
Religion Affiliation	:	Roman Catholic

FAMILY BACKGROUND

Siblings	:	Antonio B. Palmares I.
Father	:	Antonio P. Palmares Jr.
Mother (Maiden)	:	Marites U. Bajon

EDUCATIONAL QUALIFICATION

<i>Level</i>	<i>School</i>	<i>Year Graduated</i>
Elementary	Ivisan Elementary School	2016
Junior High School	Ivisan National High School	2020
Senior High School	Ivisan National High School	2022
Tertiary	Iloilo Science and Technology University	2026

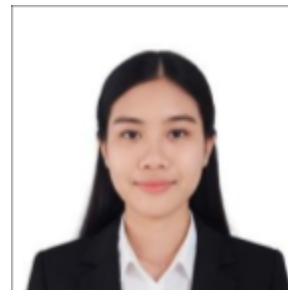
CURRICULUM VITAE

JEAN GRACE G. SAMALBURO

Home Address: Cadinglian, Oton, Iloilo 5020

Contact No.: +639129927116

Email: samalburo.jeangrace@gmail.com



PERSONAL INFORMATION

Date of Birth	:	October 21, 2001
Place of Birth	:	St. Paul's Hospital, Iloilo City, Iloilo 5000
Age	:	24
Civil Status	:	Single
Height	:	150 cm
Weight	:	42 kg
Nationality	:	Filipino
Religion Affiliation	:	Roman Catholic

FAMILY BACKGROUND

Siblings	:	Gel Mark G. Samalburo Jellie Ann G. Samalburo
Father	:	Guillermo M. Samalburo
Mother (Maiden)	:	Janet P. Gavin

EDUCATIONAL QUALIFICATION

<i>Level</i>	<i>School</i>	<i>Year Graduated</i>
Elementary	Cadinglian Batuan Elementary School	2014
Junior High School	Batuan-Cadinglian National High School	2018
Senior High School	Western Institute of Technology	2020
Tertiary	Iloilo Science and Technology University	2026

D. GRAMMARIAN CERTIFICATE



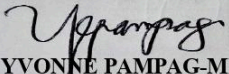
Republic of the Philippines
**Iloilo Science and Technology
University**
Burgos St., La Paz, Iloilo City, 5000 Philippines
Trunkline: (+6333) 320-7190 | Telefax: (+6333) 329-4274
<https://www.isatu.edu.ph/>
mail@isatu.edu.ph



GRAMMARIAN CERTIFICATE

This certifies that the undersigned reviewed and went through all the pages of the undergraduate thesis manuscript entitled A WEIGHTED ENSEMBLE APPROACH TO SEMANTIC SIMILARITY USING TGD, prepared by Cyra Grace B. Bedoy, Angel Mariquet E. Cachuela, Julliana Antonette D. Hurgo, Mary Angel B. Palmares, and Jean Grace G. Samalburo, in accordance with the set of structural rules that governs the composition of sentences, phrases, and words in the English Language.

This study was submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science


MS. YVONNE PAMPAG-MEDIAVILLA
Grammarian

